Annexe Corrections méthodologiques et contrôle des fuites de données

Annexe Corrections méthodologiques et contrôle des fuites de données

Cette annexe présente les corrections méthodologiques apportées à plusieurs pipelines expérimentaux développés dans le cadre de cette thèse. L'objectif principal est de documenter les procédures mises en uvre pour identifier, corriger et contrôler les risques de fuite de données (*data leakage*) susceptibles de surestimer les performances des modèles d'apprentissage automatique.

Les trois pipelines principaux concernés sont :

— la construction de l'indice de santé des sols (SHI) à partir des variables biologiques du sol ;

— la prédiction des rendements par une architecture BiLSTM–CatBoost et l'évaluation comparative par le test de Diebold–Mariano ;

— la prédiction de l'indice standardisé des précipitations (SPI) à partir de variables satellitaires et climatiques.

Un contrôle complémentaire des données est également fourni afin de vérifier automatiquement l'absence de variables susceptibles d'introduire une information provenant directement de la variable cible dans les variables explicatives.

A. Pipeline SHI : correction du risque de fuite de données

A.1. Construction de l'indice SHI

L'indice de santé des sols (*Soil Health Index*, SHI) est construit par agrégation pondérée de six indicateurs biologiques :

$$SHI = 0.4\,Cmic + 0.1\,BR + 0.1\,Xylosidase + 0.1\,\beta\text{-glucosidase} + 0.1\,Acid_phosphatase + 0.2\,N\text{-acetylgluc} \tag{1}$$

où :

— $Cmic$ représente le carbone de la biomasse microbienne ;

— BR représente la respiration basale ;

— $Xylosidase$ représente l'activité de la xylosidase ;

— β-glucosidase représente l'activité de la β-glucosidase ;

— $Acid_phosphatase$ représente l'activité de la phosphatase acide ;

— N-acetylglucosaminidase représente l'activité de la N-acétylglucosaminidase.

Les poids utilisés sont issus de la procédure de sélection décrite dans le manuscrit et permettent d'obtenir un indice composite représentant l'état biologique du sol.

A.2. Correction du protocole expérimental

La procédure corrigée impose que toutes les transformations susceptibles d'utiliser des informations statistiques soient réalisées à partir des données d'entraînement uniquement.

Le jeu de données corrigé contient :

— 881 observations au total ;
— 704 observations pour l'entraînement ;
— 177 observations pour le test ;
— une proportion de 80 % pour l'entraînement et 20 % pour le test ;
— `random_state = 42`.

Le modèle final retenu après comparaison des algorithmes est **CatBoost**.

Les performances obtenues sur le jeu de test sont :

TABLE 1 – Performances du modèle CatBoost pour la prédiction du SHI

Métrique	Valeur
R^2 test	0.850
MAE	51.2
RMSE	80.0
R^2 validation croisée	0.819

Le contrôle final du pipeline indique que les variables explicatives ne contiennent pas directement la variable cible ou une transformation de celle-ci susceptible de provoquer une fuite de données.

A.3. Code source corrigé

Le code Python correspondant est fourni dans le fichier :

`shi_lucas2018_leakage_free.py`

```
import os
import sys
import warnings

import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")  # rendu non interactif -> sauvegarde directe des figures
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.metrics import mean_absolute_error,
    mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor

import xgboost as xgb
import catboost as cb
import lightgbm as lgb
import shap

warnings.filterwarnings("ignore")

RANDOM_STATE = 42
OUTPUT_DIR = "results_SHI"
RESULTS_CSV = "SHI_LUCAS2018_results.csv"
INPUT_CSV = "LUCAS_2018_soil_functions.csv"

os.makedirs(OUTPUT_DIR, exist_ok=True)

# Global registry used by the leakage audit at the end of the
    script.
AUDIT_LOG = {}


#
    ============================================================================
# 1. DATA LOADING
#
    ============================================================================
print("=" * 70)
print("SOIL HEALTH INDEX (SHI) - LUCAS 2018 - PIPELINE
    LEAKAGE-FREE")
print("=" * 70)

if not os.path.exists(INPUT_CSV):
    sys.exit(f"[ERROR] File not found: {INPUT_CSV}")

sol_data = pd.read_csv(INPUT_CSV)

print(f"\nFile loaded: {INPUT_CSV}")
print(f"Total number of observations (LUCAS points):
    {sol_data.shape[0]}")
print(f"Total number of columns                    :
    {sol_data.shape[1]}")


#
    ----------------------------------------------------------------------------
# 1.1 Strict verification of the six biological SHI components
#
    ----------------------------------------------------------------------------
shi_components = [
```

```
    "Cmic",
    "Basal_respiration",
    "Xylosidase",
    "Beta_glucosidase",
    "Acid_phosphatase",
    "N_actylglucosaminidase",
]

missing_cols = [c for c in shi_components if c not in
   sol_data.columns]

if missing_cols:
    sys.exit(
        "[ERROR] The following SHI component columns are missing
           from the CSV: "
        f"{missing_cols}. "
        "The script does not rename or artificially create
           missing columns."
    )

print("\n[OK] All six biological/enzymatic SHI components are
   present.")


# Fixed scientific weights - NOT optimized using the dataset.
#
# SHI = 0.4*Cmic
#     + 0.1*Basal_respiration
#     + 0.1*Xylosidase
#     + 0.1*Beta_glucosidase
#     + 0.1*Acid_phosphatase
#     + 0.2*N_actylglucosaminidase

SHI_WEIGHTS = {
    "Cmic": 0.4,
    "Basal_respiration": 0.1,
    "Xylosidase": 0.1,
    "Beta_glucosidase": 0.1,
    "Acid_phosphatase": 0.1,
    "N_actylglucosaminidase": 0.2,
}

assert (
    abs(sum(SHI_WEIGHTS.values()) - 1.0) < 1e-9
), "SHI weights must sum to 1."

AUDIT_LOG["shi_weights_locked_before_split"] = True


#
   ----------------------------------------------------------------------
```

```
# 1.2 Independent predictor variables
#
    ------------------------------------------------------------------------
predictor_variables = [
    "WHC",
    "Mean_width_diameter",
    "Water_stable_aggregates",
    "MIRR",
]

missing_pred = [
    c for c in predictor_variables
    if c not in sol_data.columns
]

if missing_pred:
    sys.exit(
        f"[ERROR] Predictor variables missing from the CSV:
           {missing_pred}"
    )

print("\nSHI components:", shi_components)
print("Predictor variables:", predictor_variables)


# Explicit check:
# No SHI component can be used as a predictor.
assert set(shi_components).isdisjoint(set(predictor_variables)),
    (
    "[CRITICAL ERROR] A SHI component is present in the
       predictors -> "
    "target leakage."
)

AUDIT_LOG["shi_components_excluded_from_X"] = True

print(
    "[OK] No SHI component is present in the predictors."
)


#
    ------------------------------------------------------------------------
# 1.3 Missing-value diagnostic
#
    ------------------------------------------------------------------------
print("\n--- Missing-value rates before preprocessing ---")

all_relevant = shi_components + predictor_variables

missing_rate = (
```

```
    sol_data[all_relevant]
    .isna()
    .mean()
    .sort_values(ascending=False)
    * 100
)

for col, pct in missing_rate.items():

    flag = (
        "  <-- WARNING: high missingness"
        if pct > 50
        else ""
    )

    print(
        f"  {col:<28s}: "
        f"{pct:6.2f}% missing{flag}"
    )

if (missing_rate > 50).any():

    print(
        "\n[SCIENTIFIC WARNING] At least one predictor contains "
        "more than 50% missing values. Median imputation may
           then "
        "replace a large proportion of observations with an
           almost "
        "constant value, reducing the real predictive
           information. "
        "This limitation should be discussed in the manuscript."
    )


# The target SHI must not be artificially imputed.
# Therefore, observations missing at least one SHI component are
   removed.

sol_data = (
    sol_data
    .dropna(subset=shi_components)
    .reset_index(drop=True)
)

print(
    f"\nObservations retained after removing rows with
       incomplete "
    f"SHI components: {sol_data.shape[0]}"
)
```

```
#
    ================================================================================
# 2. SHI CONSTRUCTION
#
    ================================================================================

print("\n" + "=" * 70)
print("SHI CONSTRUCTION - FIXED SCIENTIFIC WEIGHTS")
print("=" * 70)

for var, weight in SHI_WEIGHTS.items():
    print(f"  {var:<28s} x {weight}")


sol_data["SHI"] = sum(
    SHI_WEIGHTS[var] * sol_data[var]
    for var in shi_components
)


print("\n--- SHI descriptive statistics ---")

print(f"  Minimum    : {sol_data['SHI'].min():.4f}")
print(f"  Maximum    : {sol_data['SHI'].max():.4f}")
print(f"  Mean       : {sol_data['SHI'].mean():.4f}")
print(f"  Std         : {sol_data['SHI'].std():.4f}")


#
    ================================================================================
# 3. TRAIN / TEST SPLIT
#
    ================================================================================

X = sol_data[predictor_variables].copy()
y = sol_data["SHI"].copy()

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=RANDOM_STATE,
)

print("\n" + "=" * 70)
print("TRAIN / TEST SPLIT")
print("=" * 70)

print(f"  Total observations : {X.shape[0]}")
print(f"  Training samples   : {X_train.shape[0]}")
print(f"  Test samples       : {X_test.shape[0]}")
```

```
print(f"  Predictor count    : {X.shape[1]}")

AUDIT_LOG["train_test_split_before_preprocessing"] = True


# ============================================================================
# 4. IMPUTATION - TRAINING DATA ONLY
# ============================================================================

imputer = SimpleImputer(strategy="median")

# FIT ONLY ON TRAINING DATA
imputer.fit(X_train)

AUDIT_LOG["imputer_fitted_on_train_only"] = True


X_train_imp = pd.DataFrame(
    imputer.transform(X_train),
    columns=predictor_variables,
    index=X_train.index,
)

X_test_imp = pd.DataFrame(
    imputer.transform(X_test),
    columns=predictor_variables,
    index=X_test.index,
)


# ============================================================================
# 5. STANDARDIZATION - TRAINING DATA ONLY
# ============================================================================

scaler = StandardScaler()

# FIT ONLY ON TRAINING DATA
scaler.fit(X_train_imp)

AUDIT_LOG["scaler_fitted_on_train_only"] = True


# Tree models do not require standardization.
# We therefore keep the original imputed values for the models.

X_train_final = X_train_imp
X_test_final = X_test_imp
```

```
# Apply the fitted scaler only for traceability.
_ = scaler.transform(X_train_imp)
_ = scaler.transform(X_test_imp)


#
    ================================================================================
# 6. MODEL COMPARISON USING TRAINING-ONLY CROSS-VALIDATION
#
    ================================================================================

print("\n" + "=" * 70)
print("CROSS-VALIDATION PERFORMANCE - TRAINING SET ONLY")
print("=" * 70)


models = {

    "Random Forest":
        RandomForestRegressor(
            n_estimators=100,
            random_state=RANDOM_STATE,
        ),

    "XGBoost":
        xgb.XGBRegressor(
            n_estimators=100,
            random_state=RANDOM_STATE,
            verbosity=0,
        ),

    "CatBoost":
        cb.CatBoostRegressor(
            iterations=100,
            learning_rate=0.1,
            depth=6,
            random_state=RANDOM_STATE,
            verbose=0,
        ),

    "LightGBM":
        lgb.LGBMRegressor(
            n_estimators=100,
            random_state=RANDOM_STATE,
            verbose=-1,
        ),
}
```

```
kf = KFold(
    n_splits=5,
    shuffle=True,
    random_state=RANDOM_STATE,
)

cv_results = []


for name, model in models.items():

    cv_r2 = cross_val_score(
        model,
        X_train_final,
        y_train,
        cv=kf,
        scoring="r2",
    )

    cv_mae = -cross_val_score(
        model,
        X_train_final,
        y_train,
        cv=kf,
        scoring="neg_mean_absolute_error",
    )

    cv_rmse = -cross_val_score(
        model,
        X_train_final,
        y_train,
        cv=kf,
        scoring="neg_root_mean_squared_error",
    )

    cv_results.append(
        {
            "Model": name,
            "CV_MAE_mean": cv_mae.mean(),
            "CV_MAE_std": cv_mae.std(),
            "CV_RMSE_mean": cv_rmse.mean(),
            "CV_RMSE_std": cv_rmse.std(),
            "CV_R2_mean": cv_r2.mean(),
            "CV_R2_std": cv_r2.std(),
        }
    )

    print(
        f"  {name:<15s} | "
        f"R2 = {cv_r2.mean():.4f} +/- {cv_r2.std():.4f} | "
        f"MAE = {cv_mae.mean():.4f} | "
```

```
        f"RMSE = {cv_rmse.mean():.4f}"
    )


cv_df = pd.DataFrame(cv_results)

AUDIT_LOG["model_selection_uses_cv_only"] = True


#
    ------------------------------------------------------------------------
# Select the best model using TRAINING CV only.
# The test set is not used for model selection.
#
    ------------------------------------------------------------------------

best_model_name = cv_df.loc[
    cv_df["CV_R2_mean"].idxmax(),
    "Model",
]

print(
    f"\n[SELECTION] Best model selected using training CV only: "
    f"{best_model_name}"
)


#
    ========================================================================
# 7. FINAL TRAINING AND INDEPENDENT TEST EVALUATION
#
    ========================================================================

print("\n" + "=" * 70)
print("FINAL INDEPENDENT TEST PERFORMANCE")
print("=" * 70)


fitted_models = {}

test_predictions = {
    "Observed_SHI": y_test.values
}

final_results = []


for name, model in models.items():

    # Final training using the complete training set.
    model.fit(
```

```
        X_train_final,
        y_train,
    )

    fitted_models[name] = model

    # Final test prediction.
    y_pred_test = model.predict(
        X_test_final
    )

    mae = mean_absolute_error(
        y_test,
        y_pred_test,
    )

    rmse = np.sqrt(
        mean_squared_error(
            y_test,
            y_pred_test,
        )
    )

    r2 = r2_score(
        y_test,
        y_pred_test,
    )

    final_results.append(
        {
            "Model": name,
            "MAE": mae,
            "RMSE": rmse,
            "R2": r2,
        }
    )

    print(
        f"  {name:<15s} | "
        f"MAE = {mae:.4f} | "
        f"RMSE = {rmse:.4f} | "
        f"R2 = {r2:.4f}"
    )

    short_name = {
        "Random Forest": "RF",
        "XGBoost": "XGB",
        "CatBoost": "CatBoost",
        "LightGBM": "LightGBM",
    }[name]
```

```
    test_predictions[
        f"Predicted_SHI_{short_name}"
    ] = y_pred_test


final_df = pd.DataFrame(
    final_results
)

AUDIT_LOG["test_used_only_for_final_eval"] = True


print(
    "\n--- Cross-validation results ---"
)

print(
    cv_df[
        [
            "Model",
            "CV_MAE_mean",
            "CV_RMSE_mean",
            "CV_R2_mean",
        ]
    ].to_string(index=False)
)


print(
    "\n--- Independent test results ---"
)

print(
    final_df.to_string(index=False)
)


best_model_final = fitted_models[
    best_model_name
]


#
    ============================================================================
# 8. TRUE VS PREDICTED FIGURES
#
    ============================================================================

print(
    "\nGenerating True SHI vs Predicted SHI figures..."
)
```

```
for name, model in fitted_models.items():

    y_pred_test = model.predict(
        X_test_final
    )

    fig, ax = plt.subplots(
        figsize=(6, 6)
    )

    ax.scatter(
        y_test,
        y_pred_test,
        alpha=0.5,
        color="steelblue",
    )

    lims = [
        min(
            y_test.min(),
            y_pred_test.min(),
        ),
        max(
            y_test.max(),
            y_pred_test.max(),
        ),
    ]

    ax.plot(
        lims,
        lims,
        color="red",
        lw=2,
        label="y = x (ideal)",
    )

    ax.set_xlabel(
        "True SHI"
    )

    ax.set_ylabel(
        "Predicted SHI"
    )

    ax.set_title(
        f"True vs Predicted SHI - {name}"
    )

    ax.legend()
```

```
    fig.tight_layout()

    fig.savefig(
        os.path.join(
            OUTPUT_DIR,
            f"true_vs_predicted_{name.replace(' ', '_')}.png",
        ),
        dpi=150,
    )

    plt.close(fig)


print(
    f"[OK] True vs Predicted figures saved in '{OUTPUT_DIR}/'."
)


#
   ============================================================================
# 9. SHAP ANALYSIS
#
   ============================================================================
# SHAP is computed only after the final models are locked.
# SHAP results are not used to modify the models.

print(
    "\nComputing SHAP values after model locking..."
)


for name, model in fitted_models.items():

    print(
        f"  -> SHAP for {name}..."
    )

    try:

        explainer = shap.Explainer(
            model,
            X_train_final,
        )

        shap_values = explainer(
            X_test_final
        )

    except Exception:
```

```
        explainer = shap.TreeExplainer(
            model
        )

        shap_values = explainer(
            X_test_final
        )


    # Global feature importance
    fig = plt.figure(
        figsize=(7, 5)
    )

    shap.summary_plot(
        shap_values,
        X_test_final,
        plot_type="bar",
        show=False,
    )

    plt.title(
        f"SHAP Feature Importance - {name}"
    )

    plt.tight_layout()

    plt.savefig(
        os.path.join(
            OUTPUT_DIR,
            f"shap_bar_{name.replace(' ', '_')}.png",
        ),
        dpi=150,
    )

    plt.close(fig)


    # SHAP beeswarm plot
    fig = plt.figure(
        figsize=(7, 5)
    )

    shap.summary_plot(
        shap_values,
        X_test_final,
        show=False,
    )

    plt.title(
        f"SHAP Beeswarm - {name}"
```

```
    )

    plt.tight_layout()

    plt.savefig(
        os.path.join(
            OUTPUT_DIR,
            f"shap_beeswarm_{name.replace(' ', '_')}.png",
        ),
        dpi=150,
    )

    plt.close(fig)


#
    ------------------------------------------------------------------------
# SHAP dependence plot for MIRR
#
    ------------------------------------------------------------------------

if "MIRR" in predictor_variables:

    print(
        "\nGenerating SHAP dependence plot for MIRR..."
    )

    explainer_best = shap.Explainer(
        best_model_final,
        X_train_final,
    )

    shap_values_best = explainer_best(
        X_test_final
    )

    fig = plt.figure(
        figsize=(7, 5)
    )

    shap.dependence_plot(
        "MIRR",
        shap_values_best.values,
        X_test_final,
        interaction_index=None,
        show=False,
    )

    plt.title(
        f"SHAP Dependence Plot - MIRR ({best_model_name})"
    )
```

```
    plt.tight_layout()

    plt.savefig(
        os.path.join(
            OUTPUT_DIR,
            "shap_dependence_MIRR.png",
        ),
        dpi=150,
    )

    plt.close(fig)


print(
    f"[OK] SHAP figures saved in '{OUTPUT_DIR}/'."
)

AUDIT_LOG[
    "shap_computed_after_lock_only"
] = True


#
    ============================================================================
# 10. SAVE RESULTS
#
    ============================================================================

results_export = pd.DataFrame(
    test_predictions
)

results_export.insert(
    0,
    "SHI",
    results_export["Observed_SHI"],
)

results_export.to_csv(
    RESULTS_CSV,
    index=False,
)

print(
    f"\n[OK] Final results saved to: {RESULTS_CSV}"
)


#
    ============================================================================
```

```
# 11. AUTOMATIC DATA LEAKAGE AUDIT
#
    ================================================================================

print("\n" + "=" * 70)
print("========== DATA LEAKAGE AUDIT ==========")


checks = [

    (
        "SHI component variables excluded from X",
        AUDIT_LOG.get(
            "shi_components_excluded_from_X",
            False,
        )
        and set(
            shi_components
        ).isdisjoint(
            set(predictor_variables)
        ),
    ),

    (
        "Imputer fitted only on training data",
        AUDIT_LOG.get(
            "imputer_fitted_on_train_only",
            False,
        ),
    ),

    (
        "Scaler fitted only on training data",
        AUDIT_LOG.get(
            "scaler_fitted_on_train_only",
            False,
        ),
    ),

    (
        "Model selection performed using CV only",
        AUDIT_LOG.get(
            "model_selection_uses_cv_only",
            False,
        ),
    ),

    (
        "Test set reserved for final evaluation",
        AUDIT_LOG.get(
            "test_used_only_for_final_eval",
```

```
            False,
        ),
    ),

    (
        "SHI weights fixed before final testing",
        AUDIT_LOG.get(
            "shi_weights_locked_before_split",
            False,
        ),
    ),
]


all_passed = True


for label, passed in checks:

    status = (
        "PASS"
        if passed
        else "FAIL"
    )

    if not passed:
        all_passed = False

    print(
        f"[{status}] {label}"
    )


print(
    "========================================="
)


if not all_passed:

    raise RuntimeError(
        "LEAKAGE AUDIT: FAILED - see details above."
    )


print(
    "LEAKAGE AUDIT: PASSED"
)


#
```

```
# ================================================================================
# 12. AUTOMATIC CONCLUSION
#
# ================================================================================

print(
    "\nThe SHI was constructed from six biological and enzymatic "
    "soil indicators.\n"
    "The predictive models were trained exclusively using independent "
    "soil physicochemical variables.\n"
    "The final test set was not used for preprocessing, SHI weight "
    "optimization, model selection, hyperparameter tuning, or feature "
    "selection.\n"
    "The leakage audit passed successfully."
)
```

Listing 1 – Pipeline SHI corrigé et sans fuite de données

B. Pipeline BiLSTM–CatBoost et test de Diebold–Mariano

B.1. Objectif de la correction

Une seconde vérification méthodologique a été réalisée sur le pipeline associant un réseau BiLSTM à un modèle CatBoost.

L'objectif est de garantir que les variables utilisées pendant l'entraînement ne contiennent aucune information provenant de la période ou de la valeur cible à prédire.

La séparation temporelle des observations est conservée afin d'éviter qu'une information provenant du futur soit utilisée pour prédire une observation antérieure.

B.2. Comparaison des modèles

Après correction du protocole, les prédictions obtenues par les différents modèles peuvent être comparées à partir de leurs erreurs individuelles.

Le test de Diebold–Mariano est utilisé pour déterminer si la différence de performance prédictive entre deux modèles est statistiquement significative.

Pour deux modèles A et B, on définit la différence des pertes :

$$d_t = L(e_{A,t}) - L(e_{B,t}), \tag{2}$$

où L représente la fonction de perte et $e_{A,t}$ et $e_{B,t}$ les erreurs de prédiction des deux modèles à la période t.

L'hypothèse nulle du test est :

$$H_0 : E[d_t] = 0. \tag{3}$$

Elle signifie qu'il n'existe pas de différence significative entre les performances prédictives des deux modèles considérés.

B.3. Code source corrigé

Le code correspondant au pipeline BiLSTM–CatBoost et au test de Diebold–Mariano est fourni dans :

bilstm_catboost_dm_corrected_utf8.py

```
"""
BiLSTM vs CatBoost + Diebold-Mariano test.
Leakage-free walk-forward evaluation.
Compatible with databasecor_augmented.xlsx.
"""

import pandas as pd
import numpy as np
import matplotlib

matplotlib.use("Agg")

import time
import psutil

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_percentage_error,
    r2_score,
)

import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import (
    Input,
    Dense,
    LSTM,
    Bidirectional,
    Concatenate,
    GaussianNoise,
)
from tensorflow.keras.callbacks import EarlyStopping
from catboost import CatBoostRegressor


# ============================================================
# CONFIGURATION
# ============================================================
```

```
DROP_SAVI_IF_CONSTANT = True
CLIP_NEGATIVE_SOIL_VALUES = True
SAVI_STD_THRESHOLD = 1e-6
MIN_REAL_POINTS_FOR_FOLD_METRICS = 2

file_path = "pcy_annexe/databasecor_augmented.xlsx"

df = pd.read_excel(file_path)

print("[INFO] Detected columns:")
print(df.columns.tolist())


# ============================================================
# GROWTH DURATION
# ============================================================

if "Growth_Duration" in df.columns:

    print("[INFO] Growth_Duration already exists.")

elif (
    "Growth_Start" in df.columns
    and "Growth_End" in df.columns
):

    df["Growth_Start"] = pd.to_datetime(
        df["Growth_Start"],
        errors="coerce",
    )

    df["Growth_End"] = pd.to_datetime(
        df["Growth_End"],
        errors="coerce",
    )

    df["Growth_Duration"] = (
        df["Growth_End"] - df["Growth_Start"]
    ).dt.days

    print("[INFO] Growth_Duration calculated.")

else:

    raise ValueError(
        "[ERROR] Growth_Duration or Growth_Start/Growth_End "
        "are required."
    )


# ============================================================
```

```
# YEAR COLUMN
# ================================================================

if "Year" in df.columns:

    year_col = "Year"

elif "Growth_Start" in df.columns:

    df["__Year"] = df["Growth_Start"].dt.year
    year_col = "__Year"

else:

    raise ValueError(
        "[ERROR] No usable year column found."
    )


if df[year_col].isna().any():

    raise ValueError(
        "[ERROR] Missing years detected."
    )


# ================================================================
# SYNTHETIC OBSERVATIONS
# ================================================================

if "is_synthetic" not in df.columns:

    print(
        "[WARNING] is_synthetic not found. "
        "All observations are treated as real."
    )

    df["is_synthetic"] = False


# Ensure boolean type
df["is_synthetic"] = (
    df["is_synthetic"]
    .fillna(False)
    .astype(bool)
)


# ================================================================
# FEATURES
# ================================================================
```

```
dynamic_features = [
    "Precipitation (mm)",
    "Temperature Min (řC)",
    "Temperature Max (řC)",
]


static_features_candidates = [
    "NDVI",
    "EVI",
    "SAVI",
    "Growth_Duration",
    "Area Harvested (ha)",
    "nitrogene",
    "phosphate",
    "Potassium",
    "Agricultural land(ha)",
    "Previous_Yield",
]


# ============================================================
# CHECK REQUIRED DYNAMIC FEATURES
# ============================================================

missing_dynamic = [
    c
    for c in dynamic_features
    if c not in df.columns
]

if missing_dynamic:

    raise ValueError(
        "[ERROR] Missing dynamic columns: "
        f"{missing_dynamic}"
    )


# ============================================================
# STATIC FEATURES
# ============================================================

missing_static = [
    c
    for c in static_features_candidates
    if c not in df.columns
    and c != "Previous_Yield"
]
```

```
if missing_static:

    print(
        "[INFO] Missing static columns ignored:",
        missing_static,
    )


static_features = [
    c
    for c in static_features_candidates
    if c in df.columns
    or c == "Previous_Yield"
]


target_feature = "Yield"

if target_feature not in df.columns:

    raise ValueError(
        f"[ERROR] Target column '{target_feature}' "
        "not found."
    )


# ============================================================
# MODEL PARAMETERS
# ============================================================

time_steps = 3
n_splits = 5
batch_size = 8
epochs = 100


# ============================================================
# REMOVE CONSTANT SAVI
# ============================================================

if (
    DROP_SAVI_IF_CONSTANT
    and "SAVI" in df.columns
    and df["SAVI"].std() < SAVI_STD_THRESHOLD
):

    print(
        "[INFO] SAVI is almost constant "
        "and will be removed."
    )
```

```
    if "SAVI" in static_features:

        static_features.remove("SAVI")


elif "SAVI" not in df.columns:

    print("[INFO] SAVI is not present.")


# ============================================================
# CLIP NEGATIVE SOIL VALUES
# ============================================================

if CLIP_NEGATIVE_SOIL_VALUES:

    for col in [
        "nitrogene",
        "phosphate",
        "Potassium",
    ]:

        if col in df.columns:

            n_neg = int(
                (df[col] < 0).sum()
            )

            if n_neg > 0:

                print(
                    f"[INFO] {col}: "
                    f"{n_neg} negative values clipped."
                )

                df[col] = df[col].clip(
                    lower=0
                )


print(
    "[INFO] Dynamic features:",
    dynamic_features,
)

print(
    "[INFO] Static features:",
    static_features,
)
```

```
# ============================================================
# GROUP COLUMN
# ============================================================

def infer_or_build_group_column(data):

    candidates = [
        "Group",
        "group",
        "Site",
        "site",
        "Location",
        "location",
        "Field_ID",
        "FieldId",
        "Plot",
        "plot",
        "Farm",
        "farm",
        "Station",
        "station",
        "ID",
        "id",
    ]

    for column in candidates:

        if column in data.columns:

            print(
                f"[INFO] Group column detected: "
                f"{column}"
            )

            return column

    data["__global_group"] = "ALL"

    print(
        "[WARNING] No group column found. "
        "One global time series will be used."
    )

    return "__global_group"


group_col = infer_or_build_group_column(df)


# ============================================================
# CREATE TEMPORAL SEQUENCES
```

```
# ============================================================

def create_sequences(
    data,
    dynamic_cols,
    static_cols,
    target_col,
    steps,
    group_col,
    sort_col,
):

    X_dyn = []
    X_stat = []
    y = []
    idxs = []

    for _, group_data in data.groupby(
        group_col,
        sort=False,
    ):

        if sort_col in group_data.columns:

            group_data = group_data.sort_values(
                by=sort_col
            )

        else:

            group_data = group_data.sort_index()

        if len(group_data) <= steps:
            continue

        for i in range(
            len(group_data) - steps
        ):

            X_dyn.append(
                group_data[
                    dynamic_cols
                ].iloc[
                    i:i + steps
                ].values
            )

            X_stat.append(
                group_data[
                    static_cols
                ].iloc[
```

```
                    i + steps
                ].values
            )

            y.append(
                group_data[
                    target_col
                ].iloc[
                    i + steps
                ]
            )

            idxs.append(
                group_data.index[
                    i + steps
                ]
            )

    return (
        np.asarray(X_dyn),
        np.asarray(X_stat),
        np.asarray(y),
        np.asarray(idxs),
    )


# ============================================================
# UNIQUE YEARS
# ============================================================

unique_years = np.sort(
    df[year_col].unique()
)


if len(unique_years) < n_splits + 1:

    n_splits = max(
        2,
        len(unique_years) - 1,
    )

    print(
        f"[WARNING] n_splits adjusted "
        f"to {n_splits}."
    )


if n_splits < 2:

    raise ValueError(
```

```
        "[ERROR] Not enough unique years "
        "for TimeSeriesSplit."
    )


tscv = TimeSeriesSplit(
    n_splits=n_splits
)


# ============================================================
# STORAGE
# ============================================================

bilstm_rows = []
catboost_rows = []
fold_year_ranges = []

all_y_true_real = []
all_pred_bilstm_real = []
all_pred_catboost_real = []


print(
    f"[INFO] Starting walk-forward CV "
    f"with {n_splits} folds."
)


# ============================================================
# WALK-FORWARD CROSS-VALIDATION
# ============================================================

for fold, (
    train_year_idx,
    val_year_idx,
) in enumerate(
    tscv.split(unique_years)
):

    print(
        f"\n========== "
        f"FOLD {fold + 1}/{n_splits} "
        f"=========="
    )


    # ------------------------------------------------------
    # YEARS
    # ------------------------------------------------------
```

```
    train_years = unique_years[
        train_year_idx
    ]

    val_years = unique_years[
        val_year_idx
    ]


    # ------------------------------------------------------------
    # LEAKAGE CHECK
    # ------------------------------------------------------------

    assert (
        train_years.max()
        < val_years.min()
    ), (
        f"[LEAKAGE] Fold {fold + 1}: "
        "train and validation years overlap."
    )


    fold_year_ranges.append(
        {
            "fold": fold + 1,
            "train_year_min": float(
                train_years.min()
            ),
            "train_year_max": float(
                train_years.max()
            ),
            "val_year_min": float(
                val_years.min()
            ),
            "val_year_max": float(
                val_years.max()
            ),
        }
    )


    print(
        f"[CHECK] Train "
        f"{train_years.min():.2f}-"
        f"{train_years.max():.2f} "
        f"< Val "
        f"{val_years.min():.2f}-"
        f"{val_years.max():.2f}"
    )
```

```
# ------------------------------------------------------------
# TRAIN / VALIDATION DATA
# ------------------------------------------------------------

df_train = df[
    df[year_col].isin(train_years)
].copy()

df_val = df[
    df[year_col].isin(val_years)
].copy()


n_real_val_rows = int(
    (~df_val["is_synthetic"]).sum()
)


print(
    f"[INFO] Validation rows: "
    f"{len(df_val)}, "
    f"real: {n_real_val_rows}"
)


# ------------------------------------------------------------
# SORT BEFORE PREVIOUS_YIELD
# ------------------------------------------------------------

df_train = df_train.sort_values(
    by=[group_col, year_col]
)

df_val = df_val.sort_values(
    by=[group_col, year_col]
)


# ------------------------------------------------------------
# PREVIOUS YIELD
# ------------------------------------------------------------

df_train["Previous_Yield"] = (
    df_train
    .groupby(group_col)[target_feature]
    .shift(1)
)


train_prev_mean = (
    df_train["Previous_Yield"]
```

```
    .mean()
)


if pd.isna(train_prev_mean):

    train_prev_mean = (
        df_train[target_feature]
        .mean()
    )


df_train["Previous_Yield"] = (
    df_train["Previous_Yield"]
    .fillna(train_prev_mean)
)


# ------------------------------------------------------------
# VALIDATION PREVIOUS YIELD
# ------------------------------------------------------------

df_val["Previous_Yield"] = (
    df_val
    .groupby(group_col)[target_feature]
    .shift(1)
)


df_val["Previous_Yield"] = (
    df_val["Previous_Yield"]
    .fillna(train_prev_mean)
)


# ------------------------------------------------------------
# GROWTH DURATION
# ------------------------------------------------------------

train_growth_mean = (
    df_train["Growth_Duration"]
    .mean()
)


df_train["Growth_Duration"] = (
    df_train["Growth_Duration"]
    .fillna(train_growth_mean)
)
```

```
df_val["Growth_Duration"] = (
    df_val["Growth_Duration"]
    .fillna(train_growth_mean)
)


# ------------------------------------------------------------
# SCALERS
# ------------------------------------------------------------

scaler_dyn = MinMaxScaler()
scaler_stat = MinMaxScaler()
scaler_target = MinMaxScaler()


# ------------------------------------------------------------
# FIT ONLY ON TRAINING DATA
# ------------------------------------------------------------

df_train[dynamic_features] = (
    scaler_dyn.fit_transform(
        df_train[dynamic_features]
    )
)


df_train[static_features] = (
    scaler_stat.fit_transform(
        df_train[static_features]
    )
)


df_train[[target_feature]] = (
    scaler_target.fit_transform(
        df_train[[target_feature]]
    )
)


# ------------------------------------------------------------
# TRANSFORM VALIDATION DATA
# ------------------------------------------------------------

df_val[dynamic_features] = (
    scaler_dyn.transform(
        df_val[dynamic_features]
    )
)
```

```
df_val[static_features] = (
    scaler_stat.transform(
        df_val[static_features]
    )
)


df_val[[target_feature]] = (
    scaler_target.transform(
        df_val[[target_feature]]
    )
)


# ------------------------------------------------------------
# SEQUENCES
# ------------------------------------------------------------

(
    X_dyn_train,
    X_stat_train,
    y_train,
    _,
) = create_sequences(
    df_train,
    dynamic_features,
    static_features,
    target_feature,
    time_steps,
    group_col,
    sort_col=year_col,
)


(
    X_dyn_val,
    X_stat_val,
    y_val,
    idxs_val,
) = create_sequences(
    df_val,
    dynamic_features,
    static_features,
    target_feature,
    time_steps,
    group_col,
    sort_col=year_col,
)


if (
```

```
        len(X_dyn_train) == 0
        or len(X_dyn_val) == 0
    ):

        print(
            f"[WARNING] Fold {fold + 1} skipped."
        )

        continue


    # ------------------------------------------------------------
    # REAL VALIDATION MASK
    # ------------------------------------------------------------

    real_mask_val = (
        ~df.loc[
            idxs_val,
            "is_synthetic",
        ].values
    )


    # ============================================================
    # BiLSTM MODEL
    # ============================================================

    dynamic_input = Input(
        shape=(
            time_steps,
            len(dynamic_features),
        ),
        name="dynamic_input",
    )


    x = GaussianNoise(
        0.1
    )(dynamic_input)


    x = Bidirectional(
        LSTM(
            280,
            return_sequences=True,
        )
    )(x)


    x = Bidirectional(
        LSTM(
```

```
        280,
        return_sequences=True,
    )
)(x)


x = Bidirectional(
    LSTM(
        280,
        return_sequences=False,
    )
)(x)


# ----------------------------------------------------------
# STATIC INPUT
# ----------------------------------------------------------

static_input = Input(
    shape=(
        len(static_features),
    ),
    name="static_input",
)


yb = Dense(
    100,
    activation="relu",
)(static_input)


yb = Dense(
    100,
    activation="relu",
)(yb)


# ----------------------------------------------------------
# CONCATENATION
# ----------------------------------------------------------

concat = Concatenate()(
    [x, yb]
)


# ----------------------------------------------------------
# DENSE HEAD
# ----------------------------------------------------------
```

```
z = Dense(
    100,
    activation="relu",
)(concat)


z = Dense(
    100,
    activation="relu",
)(z)


z = Dense(
    100,
    activation="relu",
)(z)


z = Dense(
    100,
    activation="relu",
)(z)


output = Dense(
    1,
    activation="linear",
)(z)


# ------------------------------------------------------------
# MODEL
# ------------------------------------------------------------

bilstm_model = Model(
    inputs=[
        dynamic_input,
        static_input,
    ],
    outputs=output,
)


bilstm_model.compile(
    optimizer="adam",
    loss="mse",
    metrics=["mae"],
)


early_stop = EarlyStopping(
```

```
    monitor="val_loss",
    patience=10,
    restore_best_weights=True,
    verbose=0,
)


# ============================================================
# BiLSTM TRAINING
# ============================================================

start_time = time.time()


start_mem = (
    psutil.Process()
    .memory_info()
    .rss
    / 1024**2
)


bilstm_model.fit(
    [
        X_dyn_train,
        X_stat_train,
    ],
    y_train,
    validation_data=(
        [
            X_dyn_val,
            X_stat_val,
        ],
        y_val,
    ),
    epochs=epochs,
    batch_size=batch_size,
    callbacks=[early_stop],
    verbose=0,
)


elapsed_bilstm = (
    time.time() - start_time
)


mem_bilstm = (
    psutil.Process()
    .memory_info()
    .rss
```

```
    / 1024**2
    - start_mem
)


# ============================================================
# BiLSTM PREDICTION
# ============================================================

pred_bilstm_scaled = (
    bilstm_model.predict(
        [
            X_dyn_val,
            X_stat_val,
        ],
        verbose=0,
    )
)


pred_bilstm = (
    scaler_target
    .inverse_transform(
        pred_bilstm_scaled
    )
    .flatten()
)


y_val_actual = (
    scaler_target
    .inverse_transform(
        y_val.reshape(-1, 1)
    )
    .flatten()
)


# ============================================================
# CATBOOST INPUT
# ============================================================

X_cat_train = np.hstack(
    [
        X_dyn_train.reshape(
            X_dyn_train.shape[0],
            -1,
        ),
        X_stat_train,
    ]
)
```

```
X_cat_val = np.hstack(
    [
        X_dyn_val.reshape(
            X_dyn_val.shape[0],
            -1,
        ),
        X_stat_val,
    ]
)


y_cat_train = y_train.ravel()


# ============================================================
# CATBOOST MODEL
# ============================================================

cat_model = CatBoostRegressor(
    iterations=100,
    depth=6,
    learning_rate=0.1,
    verbose=0,
    random_state=42,
)


# ============================================================
# CATBOOST TRAINING
# ============================================================

start_time_cb = time.time()


start_mem_cb = (
    psutil.Process()
    .memory_info()
    .rss
    / 1024**2
)


cat_model.fit(
    X_cat_train,
    y_cat_train,
)


elapsed_cb = (
```

```
    time.time()
    - start_time_cb
)


mem_cb = (
    psutil.Process()
    .memory_info()
    .rss
    / 1024**2
    - start_mem_cb
)


# ============================================================
# CATBOOST PREDICTION
# ============================================================

pred_cat_scaled = (
    cat_model.predict(
        X_cat_val
    )
)


pred_cat = (
    scaler_target
    .inverse_transform(
        pred_cat_scaled.reshape(
            -1,
            1,
        )
    )
    .flatten()
)


# ============================================================
# DIAGNOSTIC METRICS
# ============================================================

rmse_b_all = np.sqrt(
    mean_squared_error(
        y_val_actual,
        pred_bilstm,
    )
)


rmse_c_all = np.sqrt(
    mean_squared_error(
```

```
            y_val_actual,
            pred_cat,
        )
    )


    print(
        f"[DIAGNOSTIC] "
        f"BiLSTM RMSE={rmse_b_all:.2f} | "
        f"CatBoost RMSE={rmse_c_all:.2f}"
    )


    # ============================================================
    # REAL OBSERVATIONS ONLY
    # ============================================================

    n_real = int(
        real_mask_val.sum()
    )


    if (
        n_real
        >= MIN_REAL_POINTS_FOR_FOLD_METRICS
    ):

        y_real = (
            y_val_actual[
                real_mask_val
            ]
        )


        pred_b_real = (
            pred_bilstm[
                real_mask_val
            ]
        )


        pred_c_real = (
            pred_cat[
                real_mask_val
            ]
        )


        # ------------------------------------------------
        # BiLSTM METRICS
        # ------------------------------------------------
```

```
rmse_b = np.sqrt(
    mean_squared_error(
        y_real,
        pred_b_real,
    )
)


mape_b = (
    mean_absolute_percentage_error(
        y_real,
        pred_b_real,
    )
)


r2_b = (
    r2_score(
        y_real,
        pred_b_real,
    )
    if len(
        np.unique(y_real)
    ) > 1
    else np.nan
)


# ----------------------------------------------------
# CatBoost METRICS
# ----------------------------------------------------

rmse_c = np.sqrt(
    mean_squared_error(
        y_real,
        pred_c_real,
    )
)


mape_c = (
    mean_absolute_percentage_error(
        y_real,
        pred_c_real,
    )
)


r2_c = (
    r2_score(
```

```
            y_real,
            pred_c_real,
        )
        if len(
            np.unique(y_real)
        ) > 1
        else np.nan
    )


    # ------------------------------------------------------
    # DISPLAY
    # ------------------------------------------------------

    print(
        f"[BiLSTM REAL n={n_real}] "
        f"Fold {fold + 1} -> "
        f"RMSE={rmse_b:.2f} | "
        f"MAPE={mape_b * 100:.2f}% | "
        f"R2={r2_b:.4f} | "
        f"Time={elapsed_bilstm:.1f}s | "
        f"Mem={mem_bilstm:.1f}MB"
    )


    print(
        f"[CatBoost REAL n={n_real}] "
        f"Fold {fold + 1} -> "
        f"RMSE={rmse_c:.2f} | "
        f"MAPE={mape_c * 100:.2f}% | "
        f"R2={r2_c:.4f} | "
        f"Time={elapsed_cb:.1f}s | "
        f"Mem={mem_cb:.1f}MB"
    )


    # ------------------------------------------------------
    # STORE RESULTS
    # ------------------------------------------------------

    bilstm_rows.append(
        {
            "fold": fold + 1,
            "n_real_val": n_real,
            "rmse": rmse_b,
            "mape_percent": (
                mape_b * 100
            ),
            "r2": r2_b,
            "train_time_s": (
                elapsed_bilstm
```

```
                ),
                "mem_delta_mb": (
                    mem_bilstm
                ),
            }
        )


        catboost_rows.append(
            {
                "fold": fold + 1,
                "n_real_val": n_real,
                "rmse": rmse_c,
                "mape_percent": (
                    mape_c * 100
                ),
                "r2": r2_c,
                "train_time_s": (
                    elapsed_cb
                ),
                "mem_delta_mb": (
                    mem_cb
                ),
            }
        )


        # ----------------------------------------------------
        # STORE REAL BACKTEST PREDICTIONS
        # ----------------------------------------------------

        all_y_true_real.append(
            y_real
        )


        all_pred_bilstm_real.append(
            pred_b_real
        )


        all_pred_catboost_real.append(
            pred_c_real
        )


    else:

        print(
            f"[WARNING] Fold {fold + 1}: "
            f"only {n_real} real observations."
```

```
        )


# ============================================================
# DIEBOLD-MARIANO TEST
# ============================================================

def diebold_mariano(
    y_true,
    y_pred1,
    y_pred2,
):

    e1 = (
        y_true - y_pred1
    )

    e2 = (
        y_true - y_pred2
    )

    # Squared-error loss differential
    d = (
        e1**2 - e2**2
    )

    mean_d = np.mean(d)

    var_d = np.var(
        d,
        ddof=1,
    )

    if (
        len(d) < 2
        or var_d <= 0
    ):

        return np.nan

    dm_stat = (
        mean_d
        / np.sqrt(
            var_d / len(d)
        )
    )

    return dm_stat


# ============================================================
```

```
# GLOBAL BACKTEST
# ============================================================

if all_y_true_real:

    y_true_full = np.concatenate(
        all_y_true_real
    )


    pred_bilstm_full = (
        np.concatenate(
            all_pred_bilstm_real
        )
    )


    pred_cat_full = (
        np.concatenate(
            all_pred_catboost_real
        )
    )


    print(
        "[INFO] Real backtest observations: "
        f"{len(y_true_full)}"
    )


    if (
        len(y_true_full)
        >= MIN_REAL_POINTS_FOR_FOLD_METRICS
    ):

        dm_stat = diebold_mariano(
            y_true_full,
            pred_bilstm_full,
            pred_cat_full,
        )


        print(
            "\n=============================="
        )


        print(
            "DIEBOLD-MARIANO TEST"
        )
```

```
print(
    f"DM statistic: "
    f"{dm_stat:.4f}"
)


print(
    "|DM| > 1.96 indicates "
    "significance at 5%."
)


print(
    "Positive DM favors CatBoost "
    "under the defined loss."
)


print(
    "Negative DM favors BiLSTM "
    "under the defined loss."
)


print(
    "=============================="
)


# ----------------------------------------------------------
# GLOBAL RMSE
# ----------------------------------------------------------

rmse_global_b = np.sqrt(
    mean_squared_error(
        y_true_full,
        pred_bilstm_full,
    )
)


rmse_global_c = np.sqrt(
    mean_squared_error(
        y_true_full,
        pred_cat_full,
    )
)


# ----------------------------------------------------------
```

```
# GLOBAL R2
# ------------------------------------------------------

r2_global_b = (
    r2_score(
        y_true_full,
        pred_bilstm_full,
    )
    if len(
        np.unique(y_true_full)
    ) > 1
    else np.nan
)


r2_global_c = (
    r2_score(
        y_true_full,
        pred_cat_full,
    )
    if len(
        np.unique(y_true_full)
    ) > 1
    else np.nan
)


print(
    f"BiLSTM: "
    f"RMSE={rmse_global_b:.2f} "
    f"| R2={r2_global_b:.4f}"
)


print(
    f"CatBoost: "
    f"RMSE={rmse_global_c:.2f} "
    f"| R2={r2_global_c:.4f}"
)


# ======================================================
# SAVE DM RESULT
# ======================================================

pd.DataFrame(
    [
        {
            "dm_statistic": dm_stat,
            "rmse_global_bilstm": (
                rmse_global_b
```

```
                    ),
                    "r2_global_bilstm": (
                        r2_global_b
                    ),
                    "rmse_global_catboost": (
                        rmse_global_c
                    ),
                    "r2_global_catboost": (
                        r2_global_c
                    ),
                    "n_obs_total_real": (
                        len(y_true_full)
                    ),
                }
            ]
        ).to_csv(
            "dm_test_global_result.csv",
            index=False,
        )


        # ====================================================
        # SAVE BACKTEST PREDICTIONS
        # ====================================================

        pd.DataFrame(
            {
                "y_true": y_true_full,
                "pred_bilstm": (
                    pred_bilstm_full
                ),
                "pred_catboost": (
                    pred_cat_full
                ),
            }
        ).to_csv(
            "backtest_predictions_concatenated.csv",
            index=False,
        )


    else:

        print(
            "[WARNING] Not enough real observations "
            "for the DM test."
        )


else:
```

```
    print(
        "[WARNING] No real validation observations "
        "available for the DM test."
    )


# ============================================================
# SAVE FOLD RESULTS
# ============================================================

pd.DataFrame(
    bilstm_rows
).to_csv(
    "bilstm_cv_summary.csv",
    index=False,
)


pd.DataFrame(
    catboost_rows
).to_csv(
    "catboost_cv_summary.csv",
    index=False,
)


pd.DataFrame(
    fold_year_ranges
).to_csv(
    "temporal_fold_year_ranges.csv",
    index=False,
)


# ============================================================
# FINAL OUTPUT
# ============================================================

print(
    "[INFO] Fold results saved."
)


print(
    pd.DataFrame(
        fold_year_ranges
    ).to_string(
        index=False
    )
)
```

```
print(
    "[INFO] Completed."
)
```

Listing 2 – Pipeline BiLSTM–CatBoost corrigé et test de Diebold–Mariano

C. Pipeline SPI : correction de la fuite de données

C.1. Problématique

Le pipeline de prédiction du SPI a également fait l'objet d'un audit méthodologique.

Une attention particulière a été portée à la construction des variables explicatives, notamment aux variables climatiques et satellitaires calculées sur des périodes susceptibles de recouvrir la période de calcul de la cible.

L'objectif de la correction est de garantir que les variables utilisées pour la prédiction du SPI sont disponibles au moment où la prédiction est censée être réalisée.

C.2. Formulation du SPI

Dans le cadre du jeu de données considéré, la formulation utilisée après standardisation peut être exprimée sous la forme :

$$SPI = \frac{X - \mu}{\sigma}, \tag{4}$$

où X représente la quantité de précipitations considérée, μ sa moyenne calculée sur la période de référence et σ son écart-type.

C.3. Performances après correction

Les performances obtenues après correction sont présentées dans le tableau suivant.

Table 2 – Performances des modèles pour la prédiction du SPI après correction

Modèle	MSE_{train}	RMSE_{train}	R^2_{train}	MSE_{test}	RMSE_{test}	R^2_{test}
RF	0.04380	0.20927	0.9583	0.13866	0.37238	0.8205
XGBoost	0.01429	0.11956	0.9864	0.18860	0.43429	0.7559
SVR	0.18426	0.42925	0.8246	0.32597	0.57094	0.5781

Les écarts entre les coefficients de détermination d'entraînement et de test sont respectivement :

Table 3 – Écart entre les performances d'entraînement et de test

Modèle	$R^2_{train} - R^2_{test}$
RF	0.1378
XGBoost	0.2305
SVR	0.2465

Le modèle Random Forest présente ainsi le meilleur compromis entre performance prédictive et écart entre les performances d'entraînement et de test parmi les modèles évalués.

C.4. Code source corrigé

Le fichier actuellement présent dans le dossier `code` est :

`spi_prediction_corrige (3).py`

Pour faciliter la compilation LaTeX et la gestion du dépôt GitHub, il est fortement recommandé de renommer ce fichier en :

`spi_prediction_corrige.py`

Après renommage, utiliser :

```
"""
Prédiction du SPI à partir des indices satellite (composantes
   principales PC1-PC8)
--------------------------------------------------------------------------
CORRECTIONS APPORTÉES PAR RAPPORT AUX SCRIPTS ORIGINAUX
   (sylla_code.py, random_with_acp.py,
randomforest_predictions.py, randomforestonlybruit.py,
   randomizesearchcv.py,
Randomizedsearchcv_withacp.py) :

1) FUITE DE CIBLE (la plus grave) :
   SPI = (TotalPrecip - mu) / sigma  --  vérifié empiriquement :
      corr(TotalPrecip, SPI) = 1.000000
   'TotalPrecip' EST le SPI (transformation affine). L'inclure
      comme feature revient à
   donner la cible au modèle. C'est ce qui explique les Rš
      ~0.999 identiques entre RF,
   XGBoost et SVR. => TotalPrecip est retiré des features.
      Seules les variables satellite
   (PC1-PC8, dérivées de B2,B3,B4,B8, NDVI, NDWI, EVI, MSAVI)
      sont conservées, ce qui est
   d'ailleurs cohérent avec l'objectif réel : estimer le risque
      climatique par satellite,
   SANS mesure de précipitation au sol.

2) FUITE PAR NORMALISATION AVANT SPLIT :
   Dans random_with_acp.py / randomizesearchcv.py,
      MinMaxScaler.fit_transform() était
   appliqué sur X et y AVANT le train_test_split => les
      statistiques (min/max) du test
   set fuitaient dans l'entraînement. Ici, le scaler est fit
      UNIQUEMENT sur X_train.

3) 'max_features': 'auto' est déprécié/supprimé dans les
   versions récentes de scikit-learn
```

```
   pour RandomForestRegressor => remplacé par 'sqrt' / 'log2' /
      1.0.

4) La validation croisée reste un K-Fold aléatoire classique
   (pas de time series split) :
   les 200 échantillons sont des points spatiaux indépendants
      (pas une série temporelle
   séquentielle), donc RandomizedSearchCV avec
      KFold(shuffle=True) est approprié.

Après cette correction, attends-toi à un Rš nettement plus
   modeste (probablement dans une
fourchette réaliste, ex. 0.3-0.7 selon le pouvoir prédictif réel
   des indices satellite sur
le SPI) plutôt que 0.999. C'est le résultat honnête de la
   question scientifique posée.
"""

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split,
   RandomizedSearchCV, KFold
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error,
   mean_absolute_error, r2_score
import xgboost as xgb

# ELM est optionnel (package hpelm parfois instable/pas
   installé) ----------------------
try:
    from hpelm import ELM
    HPELM_AVAILABLE = True
except ImportError:
    HPELM_AVAILABLE = False
    print("[INFO] hpelm non disponible : la partie ELM sera
       ignorée.")

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

# Dossier de sortie portable : un sous-dossier "outputs" à côté
   du script,
# créé automatiquement s'il n'existe pas (fonctionne sur
   Windows/Mac/Linux).
import os
```

```
OUTPUT_DIR =
    os.path.join(os.path.dirname(os.path.abspath(__file__)),
    'outputs')
os.makedirs(OUTPUT_DIR, exist_ok=True)

#
    ============================================================================
# 1) CHARGEMENT DES DONNÉES
#
    ============================================================================
file_path = 'composantes_principales.xlsx'
data = pd.read_excel(file_path)

# --- FEATURES CORRIGÉES : uniquement les composantes
    principales satellite ----------
# TotalPrecip est EXCLU : c'est la variable qui définit SPI
    elle-même (fuite de cible).
FEATURE_COLS = ['PC1', 'PC2', 'PC3', 'PC4', 'PC5', 'PC6', 'PC7',
    'PC8']
TARGET_COL = 'SPI'

X = data[FEATURE_COLS].copy()
y = data[TARGET_COL].copy()

print(f"Features utilisées : {FEATURE_COLS}")
print(f"(TotalPrecip retiré volontairement : corr(TotalPrecip,
    SPI) = "
      f"{data['TotalPrecip'].corr(data['SPI']):.6f} -> fuite
         directe)\n")

#
    ============================================================================
# 2) SPLIT TRAIN/TEST (avant tout fit de scaler ou de modèle)
#
    ============================================================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)

#
    ============================================================================
# 3) NORMALISATION -- fit UNIQUEMENT sur le train (corrige la
    fuite #2)
#
    ============================================================================
scaler_X = StandardScaler()
X_train_scaled = pd.DataFrame(
    scaler_X.fit_transform(X_train), columns=FEATURE_COLS,
       index=X_train.index
)
X_test_scaled = pd.DataFrame(
```

```
    scaler_X.transform(X_test), columns=FEATURE_COLS,
       index=X_test.index
)
# Les modèles arbres (RF, XGBoost) n'ont pas besoin de scaling,
   mais SVR et ELM oui :
# on utilise les versions scalées pour SVR/ELM, et les versions
   non scalées (ou scalées,
# c'est équivalent pour un arbre) pour RF/XGBoost.

cv_splitter = KFold(n_splits=5, shuffle=True,
   random_state=RANDOM_STATE)

#
   ============================================================================
# 4) RANDOM FOREST
#
   ============================================================================
rf_param_grid = {
    'n_estimators': [50, 100, 200, 300],
    'max_depth': [5, 10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', 1.0],  # 'auto' déprécié ->
       corrigé
}
rf_search = RandomizedSearchCV(
    RandomForestRegressor(random_state=RANDOM_STATE),
       rf_param_grid,
    n_iter=30, cv=cv_splitter, n_jobs=-1, verbose=1,
       random_state=RANDOM_STATE,
    scoring='r2'
)
rf_search.fit(X_train, y_train)
rf_best = rf_search.best_estimator_
joblib.dump(rf_best, os.path.join(OUTPUT_DIR,
   'random_forest_model.pkl'))
print("Meilleurs paramètres RF :", rf_search.best_params_)

#
   ============================================================================
# 5) XGBOOST
#
   ============================================================================
xgb_param_grid = {
    'n_estimators': [50, 100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 6, 10],
    'subsample': [0.7, 0.8, 1.0],
    'reg_alpha': [0, 0.1, 1],
    'reg_lambda': [1, 5, 10],
}
```

```
xgb_search = RandomizedSearchCV(
    xgb.XGBRegressor(random_state=RANDOM_STATE), xgb_param_grid,
    n_iter=30, cv=cv_splitter, n_jobs=-1, verbose=1,
       random_state=RANDOM_STATE,
    scoring='r2'
)
xgb_search.fit(X_train, y_train)
xgb_best = xgb_search.best_estimator_
xgb_best.save_model(os.path.join(OUTPUT_DIR,
   'xgboost_model.json'))
print("Meilleurs paramètres XGBoost :", xgb_search.best_params_)

#
   ============================================================================
# 6) SVR (sur données scalées)
#
   ============================================================================
svr_param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': ['scale', 'auto', 0.01, 0.1, 1],
    'kernel': ['rbf', 'linear'],
    'epsilon': [0.01, 0.05, 0.1, 0.2],
}
svr_search = RandomizedSearchCV(
    SVR(), svr_param_grid, n_iter=30, cv=cv_splitter, n_jobs=-1,
       verbose=1,
    random_state=RANDOM_STATE, scoring='r2'
)
svr_search.fit(X_train_scaled, y_train)
svr_best = svr_search.best_estimator_
joblib.dump(svr_best, os.path.join(OUTPUT_DIR, 'svr_model.pkl'))
print("Meilleurs paramètres SVR :", svr_search.best_params_)

#
   ============================================================================
# 7) ELM (optionnel, sur données scalées)
#
   ============================================================================
elm_preds_test = None
if HPELM_AVAILABLE:
    elm = ELM(X_train_scaled.shape[1], 1)
    elm.add_neurons(100, 'sigm')
    elm.train(X_train_scaled.values, y_train.values.reshape(-1,
       1))
    elm_preds_test = elm.predict(X_test_scaled.values).flatten()

#
   ============================================================================
# 8) ÉVALUATION TRAIN vs TEST (permet de repérer un éventuel
   surapprentissage)
#
```

```
    ================================================================================
def evaluate(model, X_tr, y_tr, X_te, y_te, name,
    predict_fn=None):
    if predict_fn is None:
        pred_tr = model.predict(X_tr)
        pred_te = model.predict(X_te)
    else:
        pred_tr, pred_te = predict_fn()
    row = {
        'Model': name,
        'MSE (Train)': mean_squared_error(y_tr, pred_tr),
        'RMSE (Train)': np.sqrt(mean_squared_error(y_tr,
            pred_tr)),
        'R2 (Train)': r2_score(y_tr, pred_tr),
        'MSE (Test)': mean_squared_error(y_te, pred_te),
        'RMSE (Test)': np.sqrt(mean_squared_error(y_te,
            pred_te)),
        'R2 (Test)': r2_score(y_te, pred_te),
    }
    return row, pred_te

results = []
row, rf_pred_test = evaluate(rf_best, X_train, y_train, X_test,
    y_test, 'Random Forest')
results.append(row)

row, xgb_pred_test = evaluate(xgb_best, X_train, y_train,
    X_test, y_test, 'XGBoost')
results.append(row)

row, svr_pred_test = evaluate(svr_best, X_train_scaled, y_train,
    X_test_scaled, y_test, 'SVR')
results.append(row)

if HPELM_AVAILABLE:
    elm_pred_train = elm.predict(X_train_scaled.values).flatten()
    results.append({
        'Model': 'ELM',
        'MSE (Train)': mean_squared_error(y_train,
            elm_pred_train),
        'RMSE (Train)': np.sqrt(mean_squared_error(y_train,
            elm_pred_train)),
        'R2 (Train)': r2_score(y_train, elm_pred_train),
        'MSE (Test)': mean_squared_error(y_test, elm_preds_test),
        'RMSE (Test)': np.sqrt(mean_squared_error(y_test,
            elm_preds_test)),
        'R2 (Test)': r2_score(y_test, elm_preds_test),
    })

results_df = pd.DataFrame(results)
print("\n=== Résultats (sans fuite de données) ===")
```

```
print(results_df.to_string(index=False))
results_df.to_csv(os.path.join(OUTPUT_DIR,
    'resultats_spi_corriges.csv'), index=False)

#
    ============================================================================
# 9) VISUALISATION DES MÉTRIQUES DE TEST
#
    ============================================================================
metrics_to_plot = ['MSE (Test)', 'RMSE (Test)', 'MAE (Test)' if
    'MAE (Test)' in results_df else None, 'R2 (Test)']
metrics_to_plot = [m for m in metrics_to_plot if m in
    results_df.columns]

plt.figure(figsize=(12, 5))
for i, metric in enumerate(metrics_to_plot):
    plt.subplot(1, len(metrics_to_plot), i + 1)
    sns.barplot(x='Model', y=metric, data=results_df,
        palette="viridis")
    plt.title(metric)
    plt.xticks(rotation=20)
plt.tight_layout()
plt.savefig(os.path.join(OUTPUT_DIR, 'metrics_comparison.png'),
    dpi=150)
plt.close()

#
    ============================================================================
# 10) COURBES PRÉDICTIONS VS VALEURS RÉELLES (test set)
#
    ============================================================================
plt.figure(figsize=(12, 5))
plt.plot(y_test.values, label="Valeurs réelles",
    linestyle='dashed')
plt.plot(rf_pred_test, label="Random Forest", alpha=0.7)
plt.plot(xgb_pred_test, label="XGBoost", alpha=0.7)
plt.plot(svr_pred_test, label="SVR", alpha=0.7)
if HPELM_AVAILABLE:
    plt.plot(elm_preds_test, label="ELM", alpha=0.7)
plt.legend()
plt.xlabel("Échantillons (test set)")
plt.ylabel("SPI")
plt.title("Comparaison des prédictions (modèle corrigé, sans
    TotalPrecip)")
plt.savefig(os.path.join(OUTPUT_DIR, 'predictions_vs_reel.png'),
    dpi=150)
plt.close()

#
    ============================================================================
# 11) ROBUSTESSE AU BRUIT (bruit ajouté uniquement sur X_test,
```

```
    jamais sur y_test)
#
    ================================================================================
def test_robustness(models_dict, X_test_df, X_test_scaled_df,
    y_test_s, noise_levels):
    robustness_r2 = {name: [] for name in models_dict}
    for noise in noise_levels:
        Xn = X_test_df + np.random.normal(0, noise *
            X_test_df.std(), size=X_test_df.shape)
        Xn_scaled = X_test_scaled_df + np.random.normal(
            0, noise * X_test_scaled_df.std(),
                size=X_test_scaled_df.shape
        )
        for name, (model, uses_scaled) in models_dict.items():
            X_in = Xn_scaled if uses_scaled else Xn
            preds = model.predict(X_in.values if hasattr(X_in,
                'values') else X_in)
            robustness_r2[name].append(r2_score(y_test_s, preds))
    return robustness_r2

noise_levels = np.arange(0, 0.55, 0.05)
models_for_noise = {
    'Random Forest': (rf_best, False),
    'XGBoost': (xgb_best, False),
    'SVR': (svr_best, True),
}
robustness_results = test_robustness(models_for_noise, X_test,
    X_test_scaled, y_test, noise_levels)

plt.figure(figsize=(10, 5))
for name, r2_scores in robustness_results.items():
    plt.plot(noise_levels * 100, r2_scores, label=name,
        marker='o')
plt.xlabel("Niveau de bruit (%)")
plt.ylabel("Rš Score")
plt.title("Robustesse des modèles au bruit gaussien (modèle
    corrigé)")
plt.legend()
plt.grid()
plt.savefig(os.path.join(OUTPUT_DIR, 'robustesse_bruit.png'),
    dpi=150)
plt.close()

#
    ================================================================================
# 12) IMPORTANCE DES VARIABLES (SHAP)  sur le Random Forest
#
    ================================================================================
try:
    import shap
    explainer = shap.Explainer(rf_best, X_train)
```

```
    shap_values = explainer(X_test)
    shap.summary_plot(shap_values, X_test, show=False)
    plt.savefig(os.path.join(OUTPUT_DIR, 'shap_summary.png'),
       dpi=150, bbox_inches='tight')
    plt.close()
except ImportError:
    print("[INFO] shap non disponible : analyse d'importance
       ignorée.")

print(f"\nTerminé. Fichiers générés dans {OUTPUT_DIR} :")
print(" - resultats_spi_corriges.csv")
print(" - metrics_comparison.png")
print(" - predictions_vs_reel.png")
print(" - robustesse_bruit.png")
print(" - shap_summary.png (si shap installé)")
print(" - random_forest_model.pkl, xgboost_model.json,
   svr_model.pkl")
```

Listing 3 – Pipeline SPI corrigé et sans fuite de données

D. Détection des faux démarrages

Une procédure complémentaire concerne la détection des faux démarrages phénologiques à partir des données pluviométriques et agroclimatiques.

Le pipeline corrigé est disponible dans le fichier :

`version_false_start_corrige_sans_leakage.py`

```
"""
Leakage-free evaluation of hybrid LSTM/GRU/Transformer models
for false-start detection.

TWO EXPERIMENTAL MODES
----------------------

1. TEMPORAL MODE:
   - Uses temporal sequences.
   - Walk-forward / TimeSeriesSplit.
   - Tests several timesteps.

2. NO-TIMESTEP MODE:
   - No temporal sequence construction.
   - Direct tabular classification.
   - TimeSeriesSplit is still used to preserve chronology.

IMPORTANT LEAKAGE CORRECTIONS
-----------------------------

[CORRECTION 1]
```

```
StandardScaler is fitted ONLY on the training portion of each
    fold.

[CORRECTION 2]
Kendall, Pettitt and Lombard statistics are computed ONLY on
the training portion of each fold.

[CORRECTION 3]
Temporal sequences are constructed separately inside each fold.
Validation observations are never used to construct training
    sequences.

[CORRECTION 4]
StratifiedKFold(shuffle=True) has been removed for temporal data.
TimeSeriesSplit is used instead.

[CORRECTION 5]
No preprocessing is performed globally before cross-validation.

[CORRECTION 6]
The model is recreated from scratch for every fold.

[CORRECTION 7]
The no-timestep experiment operates directly on tabular
    observations.

[CORRECTION 8]
Global test statistics are never used to transform training or
validation observations.

[CORRECTION 9]
The target variable 'faux_demarage' is never included in X.

[CORRECTION 10]
Validation predictions are produced strictly from models trained
on preceding observations.

Author:
Pape El Hadji Abdoulaye Gueye
"""

# ============================================================
# IMPORTS
# ============================================================

import os
import time
import warnings

import numpy as np
import pandas as pd
```

```
import tensorflow as tf

from sklearn.model_selection import TimeSeriesSplit
from sklearn.preprocessing import StandardScaler

from sklearn.metrics import (
    roc_auc_score,
    precision_score,
    recall_score,
    f1_score,
    accuracy_score,
)

from scipy.stats import kendalltau

import matplotlib

matplotlib.use("Agg")

import matplotlib.pyplot as plt


warnings.filterwarnings("ignore")


# ============================================================
# REPRODUCIBILITY
# ============================================================

SEED = 42

np.random.seed(SEED)
tf.random.set_seed(SEED)


# ============================================================
# CONFIGURATION
# ============================================================

DATA_FILE = "donnees_avec_faux_demarage.xlsx"

TARGET = "faux_demarage"

VARIABLES_TEST = [
    "PRECTOTCORR_SUM",
    "T2M",
]

TIMESTEPS_VALUES = [
    1,
    2,
```

```
    3,
    4,
    5,
]

N_SPLITS = 5

EPOCHS = 50

BATCH_SIZE = 32

OUTPUT_DIR = "results_leakage_free"

os.makedirs(
    OUTPUT_DIR,
    exist_ok=True,
)


# ============================================================
# LOAD DATA
# ============================================================

print("\n============================================================")
print("LOADING DATA")
print("============================================================")

df = pd.read_excel(DATA_FILE)

print(
    f"[INFO] Dataset shape: {df.shape}"
)

print(
    "[INFO] Columns:"
)

print(
    df.columns.tolist()
)


# ============================================================
# BASIC VALIDATION
# ============================================================

if TARGET not in df.columns:

    raise ValueError(
        f"[ERROR] Target column '{TARGET}' is missing."
    )
```

```
missing_test_variables = [
    v
    for v in VARIABLES_TEST
    if v not in df.columns
]

if missing_test_variables:

    print(
        "[WARNING] Statistical-test variables missing:"
    )

    print(
        missing_test_variables
    )


# ============================================================
# TARGET CLEANING
# ============================================================

df = df.copy()

df = df.dropna(
    subset=[TARGET]
)

df[TARGET] = (
    pd.to_numeric(
        df[TARGET],
        errors="coerce",
    )
)

df = df.dropna(
    subset=[TARGET]
)

df[TARGET] = (
    df[TARGET]
    .astype(int)
)


unique_target = sorted(
    df[TARGET].unique()
)

print(
```

```
    "[INFO] Target classes:",
    unique_target,
)

if not set(unique_target).issubset({0, 1}):

    raise ValueError(
        "[ERROR] faux_demarage must contain "
        "binary values 0 and 1."
    )


# ============================================================
# SORT CHRONOLOGICALLY
# ============================================================

# ------------------------------------------------------------
# CORRECTION:
# The observations must be ordered chronologically BEFORE
# TimeSeriesSplit.
# ------------------------------------------------------------

if "Year" in df.columns:

    df["Year"] = pd.to_numeric(
        df["Year"],
        errors="coerce",
    )

    if df["Year"].isna().any():

        raise ValueError(
            "[ERROR] Missing values in Year."
        )

    df = df.sort_values(
        by="Year"
    ).reset_index(
        drop=True
    )

else:

    print(
        "[WARNING] 'Year' not found."
    )

    print(
        "[WARNING] Original row order will be treated "
        "as chronological order."
    )
```

```
    df = df.reset_index(
        drop=True
    )


# ============================================================
# FEATURE SELECTION
# ============================================================

# Remove target from explanatory variables.
feature_columns = [
    c
    for c in df.columns
    if c != TARGET
]


# ------------------------------------------------------------
# Remove non-numeric columns
# ------------------------------------------------------------

numeric_features = []

for column in feature_columns:

    if pd.api.types.is_numeric_dtype(
        df[column]
    ):

        numeric_features.append(
            column
        )

    else:

        print(
            f"[INFO] Non-numeric column ignored: {column}"
        )


# ------------------------------------------------------------
# Remove columns containing only missing values
# ------------------------------------------------------------

numeric_features = [
    c
    for c in numeric_features
    if not df[c].isna().all()
]
```

```
if len(numeric_features) == 0:

    raise ValueError(
        "[ERROR] No usable numeric features."
    )


print(
    f"[INFO] Number of numeric features: "
    f"{len(numeric_features)}"
)


# ============================================================
# DATA IMPUTATION FUNCTION
# ============================================================

def impute_train_validation(
    X_train,
    X_val,
):
    """
    Imputation is fitted on training data only.

    The validation set never contributes to the
    imputation statistics.
    """

    X_train = X_train.copy()

    X_val = X_val.copy()

    train_medians = (
        X_train
        .median()
    )

    X_train = (
        X_train
        .fillna(train_medians)
    )

    X_val = (
        X_val
        .fillna(train_medians)
    )

    # If a feature is entirely NaN in train,
    # use zero as a final fallback.
    X_train = X_train.fillna(0)
```

```
    X_val = X_val.fillna(0)

    return (
        X_train,
        X_val,
    )


# ============================================================
# STATISTICAL TESTS
# ============================================================

def pettitt_test(x):
    """
    Pettitt change-point test.

    This implementation is used ONLY on training data
    inside each fold.
    """

    x = np.asarray(
        x,
        dtype=float,
    )

    x = x[
        np.isfinite(x)
    ]

    n = len(x)

    if n < 3:

        return np.nan

    k_values = []

    for k in range(1, n):

        s = 0

        for i in range(k):

            for j in range(
                k,
                n,
            ):

                s += np.sign(
                    x[j] - x[i]
```

```
                )

            k_values.append(
                s
            )

    k_max = max(
        k_values,
        key=abs,
    )

    denominator = (
        n**3 - n
    )

    if denominator <= 0:

        return np.nan

    p = (
        2
        * np.exp(
            (
                -6
                * k_max**2
            )
            / denominator
        )
    )

    return float(
        min(
            p,
            1.0,
        )
    )


def lombard_test(x):
    """
    Simplified Lombard-type statistic.

    IMPORTANT:
    It is computed only from training observations.
    """

    x = np.asarray(
        x,
        dtype=float,
    )
```

```
    x = x[
        np.isfinite(x)
    ]

    n = len(x)

    if n < 3:

        return np.nan

    reference = x[0]

    differences = (
        x[1:]
        - reference
    )

    signs = np.sign(
        differences
    )

    k_values = np.arange(
        1,
        len(signs) + 1,
    )

    S = np.sum(
        k_values
        * signs
    )

    if n <= 0:

        return np.nan

    p = (
        2
        * np.exp(
            -S**2 / n
        )
    )

    return float(
        min(
            p,
            1.0,
        )
    )


def compute_training_statistical_features(
```

```
    X_train,
    X_val,
    variable_names,
):
    """
    Compute statistical-test-derived features.

    IMPORTANT:
    All statistics are calculated using X_train ONLY.

    The same training-derived statistic is then assigned
    to the corresponding variable in validation.

    This prevents validation information from entering
    the feature engineering process.
    """

    X_train = X_train.copy()

    X_val = X_val.copy()

    for var in variable_names:

        if var not in X_train.columns:

            continue

        train_values = (
            pd.to_numeric(
                X_train[var],
                errors="coerce",
            )
        )

        val_values = (
            pd.to_numeric(
                X_val[var],
                errors="coerce",
            )
        )

        valid_train = (
            train_values
            .notna()
        )

        train_clean = (
            train_values[
                valid_train
            ]
            .values
```

```
        )

        if len(train_clean) < 3:

            p_kendall = np.nan
            p_pettitt = np.nan
            p_lombard = np.nan

        else:

            time_index = np.arange(
                len(train_clean)
            )

            try:

                p_kendall = (
                    kendalltau(
                        time_index,
                        train_clean,
                    ).pvalue
                )

            except Exception:

                p_kendall = np.nan

            try:

                p_pettitt = (
                    pettitt_test(
                        train_clean
                    )
                )

            except Exception:

                p_pettitt = np.nan

            try:

                p_lombard = (
                    lombard_test(
                        train_clean
                    )
                )

            except Exception:

                p_lombard = np.nan
```

```
        # ------------------------------------------------------
        # CORRECTION:
        # The validation observations DO NOT determine
        # these values.
        # ------------------------------------------------------

        X_train[
            f"p_Kendall_{var}"
        ] = (
            p_kendall
        )

        X_val[
            f"p_Kendall_{var}"
        ] = (
            p_kendall
        )

        X_train[
            f"p_Pettitt_{var}"
        ] = (
            p_pettitt
        )

        X_val[
            f"p_Pettitt_{var}"
        ] = (
            p_pettitt
        )

        X_train[
            f"p_Lombard_{var}"
        ] = (
            p_lombard
        )

        X_val[
            f"p_Lombard_{var}"
        ] = (
            p_lombard
        )

    return (
        X_train,
        X_val,
    )


# ============================================================
# MODEL BUILDERS
# ============================================================
```

```
def build_temporal_model(
    architecture,
    timesteps,
    n_features,
):
    """
    Build LSTM/GRU/Transformer architecture.
    """

    input_layer = tf.keras.Input(
        shape=(
            timesteps,
            n_features,
        )
    )

    components = []

    if "lstm" in architecture:

        components.append(
            tf.keras.layers.LSTM(
                32
            )(
                input_layer
            )
        )

    if "gru" in architecture:

        components.append(
            tf.keras.layers.GRU(
                32
            )(
                input_layer
            )
        )

    if "transformer" in architecture:

        x_trans = (
            tf.keras.layers.Dense(
                32
            )(
                input_layer
            )
        )

        attn_out = (
            tf.keras.layers.MultiHeadAttention(
```

```
                num_heads=4,
                key_dim=8,
            )(
                x_trans,
                x_trans,
            )
        )

        trans_out = (
            tf.keras.layers.GlobalAveragePooling1D()(
                attn_out
            )
        )

        components.append(
            trans_out
        )

    if len(components) == 0:

        raise ValueError(
            "No architecture component selected."
        )

    if len(components) == 1:

        merged = components[0]

    else:

        merged = (
            tf.keras.layers.concatenate(
                components
            )
        )

    output = (
        tf.keras.layers.Dense(
            1,
            activation="sigmoid",
        )(
            merged
        )
    )

    model = tf.keras.Model(
        inputs=input_layer,
        outputs=output,
    )

    model.compile(
```

```
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=[
            "accuracy"
        ],
    )

    return model


def build_tabular_model(
    architecture,
    n_features,
):
    """
    No-timestep version.

    Directly operates on tabular features.
    """

    input_layer = tf.keras.Input(
        shape=(
            n_features,
        )
    )

    components = []

    # ------------------------------------------------------------
    # In the no-timestep version, LSTM/GRU/Transformer are
    # not mathematically appropriate because there is no
    # temporal dimension.
    #
    # Therefore the architecture becomes a dense classifier.
    # ------------------------------------------------------------

    x = (
        tf.keras.layers.Dense(
            64,
            activation="relu",
        )(
            input_layer
        )
    )

    x = (
        tf.keras.layers.Dense(
            32,
            activation="relu",
        )(
            x
```

```
        )
    )

    x = (
        tf.keras.layers.Dropout(
            0.2
        )(
            x
        )
    )

    output = (
        tf.keras.layers.Dense(
            1,
            activation="sigmoid",
        )(
            x
        )
    )

    model = tf.keras.Model(
        inputs=input_layer,
        outputs=output,
    )

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=[
            "accuracy"
        ],
    )

    return model


# ============================================================
# METRIC FUNCTION
# ============================================================

def calculate_metrics(
    y_true,
    y_prob,
):
    """
    Robust binary classification metrics.
    """

    y_true = np.asarray(
        y_true
    )
```

```
    y_prob = np.asarray(
        y_prob
    )

    y_pred = (
        y_prob >= 0.5
    ).astype(int)

    accuracy = accuracy_score(
        y_true,
        y_pred,
    )

    precision = precision_score(
        y_true,
        y_pred,
        zero_division=0,
    )

    recall = recall_score(
        y_true,
        y_pred,
        zero_division=0,
    )

    f1 = f1_score(
        y_true,
        y_pred,
        zero_division=0,
    )

    if len(
        np.unique(y_true)
    ) == 2:

        auc = roc_auc_score(
            y_true,
            y_prob,
        )

    else:

        auc = np.nan

    return {
        "Accuracy": accuracy,
        "AUC": auc,
        "Precision": precision,
        "Recall": recall,
        "F1": f1,
```

```
    }


# ============================================================
# PREPARE BASE DATA
# ============================================================

X_base = (
    df[
        numeric_features
    ]
    .copy()
)

y_base = (
    df[TARGET]
    .values
)


# ============================================================
# TEMPORAL EXPERIMENT
# ============================================================

def run_temporal_experiment(
    timesteps,
    architecture,
):
    """
    Leakage-free temporal experiment.

    Important:
    The complete dataset is NEVER scaled before CV.
    """

    print(
        "\n"
        + "=" * 70
    )

    print(
        f"TEMPORAL EXPERIMENT | "
        f"timesteps={timesteps} | "
        f"architecture={architecture}"
    )

    print(
        "=" * 70
    )

    results = []
```

```
    # ------------------------------------------------------------
    # TimeSeriesSplit operates on chronological rows.
    # ------------------------------------------------------------

    tscv = TimeSeriesSplit(
        n_splits=N_SPLITS
    )

    for fold, (
        train_idx,
        val_idx,
    ) in enumerate(
        tscv.split(
            X_base
        ),
        start=1,
    ):

        print(
            f"\n[FOLD {fold}/{N_SPLITS}]"
        )

        # ------------------------------------------------------------
        # RAW SPLIT
        # ------------------------------------------------------------

        X_train_raw = (
            X_base
            .iloc[
                train_idx
            ]
            .copy()
        )

        X_val_raw = (
            X_base
            .iloc[
                val_idx
            ]
            .copy()
        )

        y_train = (
            y_base[
                train_idx
            ]
        )

        y_val = (
            y_base[
```

```
            val_idx
        ]
    )

    print(
        f"Train size: {len(train_idx)}"
    )

    print(
        f"Validation size: {len(val_idx)}"
    )

    # ----------------------------------------------------
    # CLASS CHECK
    # ----------------------------------------------------

    if len(
        np.unique(y_train)
    ) < 2:

        print(
            "[WARNING] Training fold contains "
            "only one class. Fold skipped."
        )

        continue

    if len(
        np.unique(y_val)
    ) < 2:

        print(
            "[WARNING] Validation fold contains "
            "only one class. AUC will be NaN."
        )

    # ----------------------------------------------------
    # STATISTICAL FEATURES
    # ----------------------------------------------------

    X_train_raw, X_val_raw = (
        compute_training_statistical_features(
            X_train_raw,
            X_val_raw,
            VARIABLES_TEST,
        )
    )

    # ----------------------------------------------------
    # IMPUTATION
    # ----------------------------------------------------
```

```
X_train_raw, X_val_raw = (
    impute_train_validation(
        X_train_raw,
        X_val_raw,
    )
)

# ------------------------------------------------------
# SCALING
#
# CORRECTION:
# scaler.fit() ONLY on training data.
# ------------------------------------------------------

scaler = StandardScaler()

X_train_scaled = (
    scaler.fit_transform(
        X_train_raw
    )
)

X_val_scaled = (
    scaler.transform(
        X_val_raw
    )
)

# ------------------------------------------------------
# BUILD TEMPORAL SEQUENCES
#
# CORRECTION:
# Sequences are created separately after the fold split.
# ------------------------------------------------------

X_train_seq = []

y_train_seq = []

for i in range(
    timesteps,
    len(
        X_train_scaled
    ),
):

    X_train_seq.append(
        X_train_scaled[
            i - timesteps:i
        ]
```

```
        )

        y_train_seq.append(
            y_train[i]
        )

    X_val_seq = []

    y_val_seq = []

    for i in range(
        timesteps,
        len(
            X_val_scaled
        ),
    ):

        X_val_seq.append(
            X_val_scaled[
                i - timesteps:i
            ]
        )

        y_val_seq.append(
            y_val[i]
        )

    if (
        len(X_train_seq) == 0
        or len(X_val_seq) == 0
    ):

        print(
            "[WARNING] Not enough observations "
            "for this timestep."
        )

        continue

    X_train_seq = np.asarray(
        X_train_seq,
        dtype=np.float32,
    )

    X_val_seq = np.asarray(
        X_val_seq,
        dtype=np.float32,
    )

    y_train_seq = np.asarray(
        y_train_seq,
```

```
    dtype=np.float32,
)

y_val_seq = np.asarray(
    y_val_seq,
    dtype=np.float32,
)

# ------------------------------------------------------
# BUILD MODEL
# ------------------------------------------------------

model = build_temporal_model(
    architecture,
    timesteps,
    X_train_seq.shape[2],
)

# ------------------------------------------------------
# EARLY STOPPING
# ------------------------------------------------------

early_stop = (
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,
        restore_best_weights=True,
    )
)

# ------------------------------------------------------
# TRAIN
# ------------------------------------------------------

start = time.time()

model.fit(
    X_train_seq,
    y_train_seq,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_seq,
        y_val_seq,
    ),
    callbacks=[
        early_stop
    ],
    verbose=0,
)
```

```
        elapsed = (
            time.time()
            - start
        )

        # ----------------------------------------------------
        # PREDICTION
        # ----------------------------------------------------

        y_prob = (
            model.predict(
                X_val_seq,
                verbose=0,
            )
            .flatten()
        )

        metrics = calculate_metrics(
            y_val_seq,
            y_prob,
        )

        metrics[
            "Fold"
        ] = fold

        metrics[
            "Timesteps"
        ] = timesteps

        metrics[
            "Architecture"
        ] = architecture

        metrics[
            "Train_Time_s"
        ] = elapsed

        metrics[
            "N_Train"
        ] = len(
            y_train_seq
        )

        metrics[
            "N_Validation"
        ] = len(
            y_val_seq
        )

        results.append(
```

```
            metrics
        )

        print(
            f"Accuracy={metrics['Accuracy']:.4f} | "
            f"AUC={metrics['AUC']:.4f} | "
            f"F1={metrics['F1']:.4f}"
        )

        # ----------------------------------------------------
        # CLEAN MODEL
        # ----------------------------------------------------

        tf.keras.backend.clear_session()

    return results


# ============================================================
# NO-TIMESTEP EXPERIMENT
# ============================================================

def run_no_timestep_experiment():
    """
    Leakage-free tabular experiment.

    No sequence is created.
    No LSTM/GRU/Transformer temporal dimension is used.

    This provides a clean baseline for comparison.
    """

    print(
        "\n"
        + "=" * 70
    )

    print(
        "NO-TIMESTEP EXPERIMENT"
    )

    print(
        "=" * 70
    )

    results = []

    tscv = TimeSeriesSplit(
        n_splits=N_SPLITS
    )
```

```
    for fold, (
        train_idx,
        val_idx,
    ) in enumerate(
        tscv.split(
            X_base
        ),
        start=1,
    ):

        print(
            f"\n[FOLD {fold}/{N_SPLITS}]"
        )

        X_train_raw = (
            X_base
            .iloc[
                train_idx
            ]
            .copy()
        )

        X_val_raw = (
            X_base
            .iloc[
                val_idx
            ]
            .copy()
        )

        y_train = (
            y_base[
                train_idx
            ]
        )

        y_val = (
            y_base[
                val_idx
            ]
        )

        if len(
            np.unique(y_train)
        ) < 2:

            print(
                "[WARNING] Training fold has "
                "only one class. Skipped."
            )
```

```
        continue

    # ----------------------------------------------------
    # STATISTICAL TESTS
    # ----------------------------------------------------

    X_train_raw, X_val_raw = (
        compute_training_statistical_features(
            X_train_raw,
            X_val_raw,
            VARIABLES_TEST,
        )
    )

    # ----------------------------------------------------
    # IMPUTATION
    # ----------------------------------------------------

    X_train_raw, X_val_raw = (
        impute_train_validation(
            X_train_raw,
            X_val_raw,
        )
    )

    # ----------------------------------------------------
    # SCALING
    # ----------------------------------------------------

    scaler = StandardScaler()

    X_train_scaled = (
        scaler.fit_transform(
            X_train_raw
        )
    )

    X_val_scaled = (
        scaler.transform(
            X_val_raw
        )
    )

    # ----------------------------------------------------
    # BUILD TABULAR MODEL
    # ----------------------------------------------------

    model = build_tabular_model(
        architecture="MLP",
        n_features=X_train_scaled.shape[1],
    )
```

```
early_stop = (
    tf.keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=10,
        restore_best_weights=True,
    )
)

start = time.time()

model.fit(
    X_train_scaled,
    y_train,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(
        X_val_scaled,
        y_val,
    ),
    callbacks=[
        early_stop
    ],
    verbose=0,
)

elapsed = (
    time.time()
    - start
)

y_prob = (
    model.predict(
        X_val_scaled,
        verbose=0,
    )
    .flatten()
)

metrics = calculate_metrics(
    y_val,
    y_prob,
)

metrics[
    "Fold"
] = fold

metrics[
    "Timesteps"
] = 0
```

```
        metrics[
            "Architecture"
        ] = "MLP_no_timestep"

        metrics[
            "Train_Time_s"
        ] = elapsed

        metrics[
            "N_Train"
        ] = len(
            y_train
        )

        metrics[
            "N_Validation"
        ] = len(
            y_val
        )

        results.append(
            metrics
        )

        print(
            f"Accuracy={metrics['Accuracy']:.4f} | "
            f"AUC={metrics['AUC']:.4f} | "
            f"F1={metrics['F1']:.4f}"
        )

        tf.keras.backend.clear_session()

    return results


# ============================================================
# RUN TEMPORAL MODELS
# ============================================================

all_temporal_results = []

model_configs = {

    "LSTM": [
        "lstm"
    ],

    "LSTM_GRU": [
        "lstm",
        "gru",
```

```
    ],

    "LSTM_Transformer": [
        "lstm",
        "transformer",
    ],

    "GRU_Transformer": [
        "gru",
        "transformer",
    ],

    "LSTM_GRU_Transformer": [
        "lstm",
        "gru",
        "transformer",
    ],
}


for timesteps in TIMESTEPS_VALUES:

    for architecture_name, architecture in (
        model_configs.items()
    ):

        fold_results = (
            run_temporal_experiment(
                timesteps,
                architecture,
            )
        )

        all_temporal_results.extend(
            fold_results
        )


# ============================================================
# RUN NO-TIMESTEP MODEL
# ============================================================

no_timestep_results = (
    run_no_timestep_experiment()
)


# ============================================================
# COMBINE RESULTS
# ============================================================
```

```
all_results = (
    all_temporal_results
    + no_timestep_results
)


results_df = pd.DataFrame(
    all_results
)


if len(results_df) == 0:

    raise RuntimeError(
        "[ERROR] No valid results were generated."
    )


# ============================================================
# SAVE FOLD RESULTS
# ============================================================

fold_results_path = os.path.join(
    OUTPUT_DIR,
    "leakage_free_fold_results.csv",
)

results_df.to_csv(
    fold_results_path,
    index=False,
)


# ============================================================
# AGGREGATED RESULTS
# ============================================================

summary_df = (
    results_df
    .groupby(
        [
            "Timesteps",
            "Architecture",
        ]
    )
    .agg(
        Accuracy_mean=(
            "Accuracy",
            "mean",
        ),
        Accuracy_std=(
```

```
            "Accuracy",
            "std",
        ),
        AUC_mean=(
            "AUC",
            "mean",
        ),
        AUC_std=(
            "AUC",
            "std",
        ),
        Precision_mean=(
            "Precision",
            "mean",
        ),
        Recall_mean=(
            "Recall",
            "mean",
        ),
        F1_mean=(
            "F1",
            "mean",
        ),
        F1_std=(
            "F1",
            "std",
        ),
        Train_Time_mean=(
            "Train_Time_s",
            "mean",
        ),
    )
    .reset_index()
)


# ============================================================
# SAVE SUMMARY
# ============================================================

summary_path = os.path.join(
    OUTPUT_DIR,
    "leakage_free_summary.csv",
)

summary_df.to_csv(
    summary_path,
    index=False,
)
```

```
# ============================================================
# DISPLAY SUMMARY
# ============================================================

print(
    "\n"
    + "=" * 80
)

print(
    "LEAKAGE-FREE EXPERIMENTAL SUMMARY"
)

print(
    "=" * 80
)

print(
    summary_df.to_string(
        index=False
    )
)


# ============================================================
# SAVE CONFIGURATION
# ============================================================

config_df = pd.DataFrame(
    [
        {
            "data_file": DATA_FILE,
            "target": TARGET,
            "n_features_original": len(
                numeric_features
            ),
            "n_splits": N_SPLITS,
            "epochs": EPOCHS,
            "batch_size": BATCH_SIZE,
            "timesteps_tested": str(
                TIMESTEPS_VALUES
            ),
            "validation_method": (
                "TimeSeriesSplit"
            ),
            "scaling": (
                "fit_on_training_fold_only"
            ),
            "statistical_tests": (
                "training_fold_only"
            ),
```

```
            "sequence_creation": (
                "inside_each_fold"
            ),
            "shuffle": False,
            "no_timestep_version": True,
        }
    ]
)


config_df.to_csv(
    os.path.join(
        OUTPUT_DIR,
        "leakage_free_configuration.csv",
    ),
    index=False,
)


# ============================================================
# FINAL MESSAGE
# ============================================================

print(
    "\n============================================================"
)

print(
    "[INFO] Leakage-free evaluation completed."
)

print(
    f"[INFO] Fold results: {fold_results_path}"
)

print(
    f"[INFO] Summary: {summary_path}"
)

print(
    "[INFO] The no-timestep baseline is included."
)

print(
    "============================================================"
)
```

Listing 4 – Pipeline corrigé de détection des faux démarrages sans fuite de données

E. Contrôle automatique des fuites de données

Un script indépendant a également été développé afin de contrôler les variables utilisées dans les différents pipelines.

Le fichier est :

`Controler_data_leakage.py`

Il permet notamment de rechercher les variables susceptibles d'être directement ou indirectement liées à la variable cible et de vérifier la cohérence entre les variables explicatives et la cible.

```
from pathlib import Path

base = Path("/mnt/data/annexe_shi_spi_leakage_free")
base.mkdir(parents=True, exist_ok=True)

tex = r"""%
% ============================================================================
% ANNEXE  Corrections méthodologiques des pipelines SHI et SPI
% Document destiné au dépôt GitHub avec les scripts Python corrigés.
%
% Compilation recommandée : XeLaTeX ou LuaLaTeX
% Dépendances : book, babel, geometry, listings, xcolor, float, booktabs,
%               amsmath, amssymb, enumitem, hyperref
%
% Arborescence recommandée :
%   annexe_shi_spi_leakage_free.tex
%   code/
%       shi_lucas2018_leakage_free.py
%       spi_satellite_leakage_free.py
%
% ============================================================================

\documentclass[11pt,a4paper]{report}

\usepackage[french]{babel}
\usepackage[T1]{fontenc}
\usepackage[utf8]{inputenc}
\usepackage[a4paper,margin=2.5cm]{geometry}
\usepackage{amsmath,amssymb}
\usepackage{booktabs}
\usepackage{float}
\usepackage{enumitem}
\usepackage{xcolor}
\usepackage{listings}
\usepackage{hyperref}

\hypersetup{
    colorlinks=true,
```

```
    linkcolor=black,
    urlcolor=blue,
    citecolor=black
}

\lstdefinestyle{pythonstyle}{
    language=Python,
    basicstyle=\ttfamily\footnotesize,
    keywordstyle=\color{blue}\bfseries,
    commentstyle=\color{gray}\itshape,
    stringstyle=\color{teal},
    numbers=left,
    numberstyle=\tiny\color{gray},
    stepnumber=1,
    numbersep=8pt,
    frame=single,
    breaklines=true,
    breakatwhitespace=false,
    showstringspaces=false,
    tabsize=4,
    captionpos=b,
    columns=fullflexible,
    keepspaces=true
}

\title{Corrections méthodologiques des pipelines\\
SHI et SPI}
\author{Pape El Hadji Abdoulaye Gueye}
\date{\today}

\begin{document}

\maketitle

\tableofcontents

%
   ================================================================================
\chapter{Correction du pipeline de construction et de prédiction
   du
\textit{Soil Health Index} (SHI)}
\label{annexe:shi-leakage-free}

Cette annexe documente les corrections méthodologiques apportées
   au pipeline
de construction et de prédiction du \textit{Soil Health Index}
   (SHI) à partir
du jeu de données LUCAS 2018, en réponse à plusieurs sources de
   fuite de
données (\textit{data leakage}) identifiées dans une version
   antérieure du
```

```
script.

\section{Problèmes identifiés dans la version initiale}

La version initiale du pipeline présentait quatre sources
   potentielles de
fuite de données (\textit{data leakage}) :

\begin{enumerate}
    \item l'imputation des valeurs manquantes par la médiane
       était calculée
    sur l'ensemble du jeu de données \emph{avant} la séparation
       entre les
    ensembles d'entraînement et de test ;

    \item les pondérations du SHI étaient optimisées par
       recherche exhaustive
    (\textit{grid search}) en utilisant le score $R^2$ calculé
       sur le jeu de
    test. Cette procédure revenait à sélectionner la définition
       de la variable
    cible en fonction des données destinées à l'évaluation
       finale ;

    \item certaines composantes biologiques ayant servi à
       construire le SHI,
    notamment \textit{Cmic}, \textit{Xylosidase} et
    \textit{Basal\_respiration}, étaient réintroduites comme
       variables
    prédictives, créant ainsi une fuite conceptuelle directe
       entre la cible
    et les prédicteurs ;

    \item le choix du meilleur modèle parmi les algorithmes
       évalués reposait
    sur les performances mesurées sur le jeu de test plutôt que
       sur une
    validation croisée réalisée exclusivement sur l'ensemble
       d'entraînement.
\end{enumerate}

\section{Corrections apportées}

Le pipeline a été intégralement révisé afin de garantir une
   séparation stricte
entre les données d'entraînement et les données d'évaluation,
   conformément aux
bonnes pratiques de validation en apprentissage automatique
   appliqué aux
sciences du sol.
```

```
Les principales corrections sont les suivantes :

\begin{itemize}
    \item les pondérations du SHI,
    \[
    \mathrm{SHI}
    =
    0.4\,\mathrm{Cmic}
    +0.1\,\mathrm{Basal\_respiration}
    +0.1\,\mathrm{Xylosidase}
    +0.1\,\mathrm{Beta\_glucosidase}
    +0.1\,\mathrm{Acid\_phosphatase}
    +0.2\,\mathrm{N\_acetylglucosaminidase},
    \]
    ont été fixées \textit{a priori} sur la base de la
       définition retenue
    pour l'indice et ne sont plus ré-optimisées à partir du jeu
       de test ;

    \item l'imputation des valeurs manquantes, réalisée avec
    \texttt{SimpleImputer} selon une stratégie par médiane, et la
    standardisation, réalisée avec \texttt{StandardScaler}, sont
       ajustées
    (\texttt{fit}) exclusivement sur l'ensemble d'entraînement,
       puis
    appliquées (\texttt{transform}) à l'ensemble de test ;

    \item les variables prédictives, à savoir \textit{WHC},
    \textit{Mean\_width\_diameter},
       \textit{Water\_stable\_aggregates} et
    \textit{MIRR}, sont vérifiées comme étant disjointes des
       composantes
    biologiques utilisées pour construire le SHI au moyen d'une
       assertion
    automatique :
\begin{verbatim}
assert set(shi_components).isdisjoint(set(predictor_variables))
\end{verbatim}

    \item la sélection du meilleur modèle parmi Random Forest,
       XGBoost,
    CatBoost et LightGBM est effectuée au moyen d'une validation
       croisée à
    5 plis (\textit{5-fold cross-validation}) réalisée
       exclusivement sur
    l'ensemble d'entraînement. Le jeu de test n'intervient
       qu'une seule fois,
    lors de l'évaluation finale du modèle retenu ;

    \item l'analyse d'explicabilité SHAP est réalisée après le
       verrouillage
```

```
    des modèles finaux et n'est pas utilisée pour modifier
       ultérieurement
    les modèles, les hyperparamètres ou les variables
       prédictives ;

    \item un module d'audit automatique (\textit{leakage audit})
       vérifie,
    à l'issue de l'exécution, le respect des principaux critères
       d'absence de
    fuite de données.
\end{itemize}

\section{Résultats obtenus après correction}

Sur les 881 observations exploitables du jeu de données LUCAS
   2018, dont
704 observations utilisées pour l'entraînement et 177 pour le
   test, selon
une séparation aléatoire de 80/20 avec
   \texttt{random\_state=42}, le modèle
CatBoost obtient les meilleures performances sur le jeu de test
   indépendant,
avec un coefficient de détermination de $R^2=0.850$, une MAE de
   $51.2$ et une
RMSE de $80.0$.

Ces performances sont cohérentes avec celles obtenues lors de la
   validation
croisée sur l'ensemble d'entraînement, pour laquelle CatBoost
   atteint
$R^2_{\mathrm{CV}}=0.819$. Cette cohérence entre la validation
   croisée et
l'évaluation indépendante renforce la crédibilité des résultats
   obtenus
après correction du pipeline.

L'audit automatisé de fuite de données confirme le respect des
   contraintes
méthodologiques définies précédemment et retourne le statut
\texttt{LEAKAGE AUDIT: PASSED}.

\section{Code source complet}
\label{annexe:shi-leakage-free-code}

Le code source complet du pipeline corrigé est fourni séparément
   dans le
répertoire \texttt{code/}. Le script recommandé est
\texttt{shi\_lucas2018\_leakage\_free.py}.

Pour l'inclure directement dans une version destinée à la thèse,
   utiliser :
```

```
\begin{verbatim}
\lstinputlisting[style=pythonstyle,
caption={Pipeline SHI corrigé et sans fuite de données}]
{code/shi_lucas2018_leakage_free.py}
\end{verbatim}

Le fichier Python doit donc être placé dans le sous-dossier
\texttt{code/} du projet LaTeX. Cette organisation permet de
   conserver
séparément le texte méthodologique et le code exécutable.

\section{Référencement du dépôt de code}

Le dépôt GitHub associé doit être indiqué ici lorsque son URL
   définitive
est disponible :

\begin{center}
\texttt{[URL DU DÉPÔT GITHUB À RENSEIGNER]}
\end{center}

%
   ==========================================================================
\chapter{Pipeline de comparaison BiLSTM--CatBoost et test de
Diebold--Mariano}
\label{annexe:bilstm-catboost-dm}

Cette annexe présente le pipeline expérimental utilisé pour
   comparer les
performances du modèle BiLSTM et du modèle CatBoost dans un
   cadre de
validation temporelle sans fuite de données. Le protocole repose
   sur une
validation croisée \textit{walk-forward} utilisant
\texttt{TimeSeriesSplit}, de manière à garantir que les
   observations
utilisées pour l'entraînement précèdent strictement les
   observations
utilisées pour la validation.

Une attention particulière est portée à la séparation entre les
   observations
réelles et les observations synthétiques. Les observations
   synthétiques
peuvent être utilisées pendant l'entraînement afin de densifier
   les séries
temporelles, mais elles sont systématiquement exclues des
   métriques finales
et du test de Diebold--Mariano. Ainsi, les performances
   rapportées pour la
```

```
comparaison des modèles reposent exclusivement sur des
   observations réelles.

Pour chaque pli temporel, les variables de transformation sont
   ajustées
exclusivement sur les données d'entraînement. Les paramètres des
\texttt{MinMaxScaler} utilisés pour les variables dynamiques,
   les variables
statiques et la variable cible sont donc estimés sur le
   sous-ensemble
d'entraînement, puis appliqués au sous-ensemble de validation.
   De même, la
variable \texttt{Previous\_Yield} est reconstruite
   indépendamment dans
chaque pli afin d'éviter l'utilisation d'informations futures.

Le modèle BiLSTM exploite une séquence temporelle de longueur
   $T=3$ et
combine les variables dynamiques avec les variables statiques.
   CatBoost
reçoit les mêmes séquences temporelles aplaties et les mêmes
   variables
statiques, ce qui permet une comparaison fondée sur les mêmes
   observations
et les mêmes informations disponibles.

Les performances sont évaluées à l'aide du RMSE, du MAPE et du
   coefficient
de détermination $R^2$. Enfin, les erreurs de prédiction hors
   échantillon
obtenues sur les observations réelles de l'ensemble des plis sont
concaténées afin d'effectuer le test de Diebold--Mariano. Ce
   test permet
d'évaluer si la différence de précision prédictive entre le
   BiLSTM et
CatBoost est statistiquement significative.

\section{Code source}

Le script Python correspondant doit être placé dans le répertoire
\texttt{code/}, par exemple sous le nom
\texttt{bilstm\_catboost\_dm\_leakage\_free.py}.

%
   ==========================================================================
\chapter{Correction d'une fuite de données dans la Contribution 3
(prédiction du SPI)}
\label{annexe:fuite-spi}

\section{Contexte et diagnostic du problème}
```

```
Une relecture méthodologique a posteriori de la Contribution 3 a
   permis
d'identifier une fuite de données (\textit{data leakage})
   affectant
l'ensemble des modèles présentés initialement (Random Forest,
   XGBoost,
SVR, ELM). Cette annexe documente le diagnostic, la correction
   apportée
et les résultats obtenus après correction, dans un souci de
   transparence
méthodologique.

Le Standardized Precipitation Index (SPI) est, dans le pipeline
   considéré,
une transformation affine de la précipitation cumulée sur la
   période
étudiée :
\[
SPI=\frac{X-\mu}{\sigma},
\]
où $X$ est la précipitation totale observée
(\textit{TotalPrecip} dans le jeu de données), et $\mu$,
   $\sigma$ sont
respectivement la moyenne et l'écart-type historiques de cette
précipitation.

Cette relation a été vérifiée empiriquement sur le jeu de
   données utilisé :
la corrélation entre \textit{TotalPrecip} et le SPI est de
   $1.000000$, et
l'écart absolu entre le SPI observé et le score-$z$ de
\textit{TotalPrecip} est de l'ordre de $10^{-15}$, ce qui
   correspond au
bruit de calcul en virgule flottante. Cette vérification
   confirme qu'il
s'agit, dans ce jeu de données, d'une identité mathématique et
   non d'une
simple corrélation forte.

Or, dans la chaîne de modélisation initiale,
   \textit{TotalPrecip} figurait
parmi les variables explicatives utilisées pour prédire le SPI.
   Cela revient
à fournir au modèle une transformation déterministe de la cible
   elle-même.
Les algorithmes peuvent alors redécouvrir la relation
$(X-\mu)/\sigma$ au lieu d'apprendre une relation prédictive
   indépendante.

Ce phénomène explique les performances quasi parfaites rapportées
initialement : $R^2=0.9991$ pour Random Forest, $R^2=0.9984$
```

```
   pour XGBoost
et $R^2=0.9971$ pour SVR, ainsi que la domination écrasante de
\textit{TotalPrecip} dans l'analyse SHAP initiale.

Une fuite secondaire, de moindre ampleur, a également été
   identifiée dans
certaines variantes du pipeline : la normalisation
(\texttt{MinMaxScaler}) était ajustée (\texttt{fit\_transform})
   sur
l'ensemble des données \textbf{avant} la séparation train/test,
   permettant
aux statistiques (min/max) de l'ensemble de test de fuiter dans
l'entraînement.

\section{Correction méthodologique apportée}

Les corrections suivantes ont été appliquées :

\begin{enumerate}
    \item \textbf{Suppression de \textit{TotalPrecip} des
       variables
    explicatives.} Seules les variables dérivées de l'imagerie
       satellite
    (composantes principales PC1 à PC8, issues des bandes
       spectrales B2,
    B3, B4, B8 et des indices NDVI, NDWI, MSAVI, EVI) sont
       conservées comme
    prédicteurs. Ce choix est cohérent avec l'objectif
       scientifique réel de
    la contribution : estimer le risque climatique à partir de
    l'observation satellite seule, dans les zones où la mesure
       directe de
    précipitation au sol est absente ou peu fiable.

    \item \textbf{Ajustement du normaliseur uniquement sur
       l'ensemble
    d'entraînement}, puis application (\texttt{transform}) à
       l'ensemble
    de test, afin d'éliminer toute fuite d'information
       statistique du test
    vers l'entraînement.

    \item \textbf{Validation croisée K-Fold aléatoire} (5 plis,
    \texttt{shuffle=True}) pour l'optimisation des
       hyperparamètres via
    \textit{RandomizedSearchCV}, les 200 échantillons
       correspondant à des
    points spatiaux indépendants et non à une série temporelle
       séquentielle.
\end{enumerate}
```

```
\section{Résultats après correction}

Le Tableau~\ref{tab:performance_spi_corrige} présente les
   performances
obtenues après suppression de la fuite de données.

\begin{table}[H]
\centering
\small
\caption{Performance des modèles en entraînement et test, après
   correction
de la fuite de données}
\label{tab:performance_spi_corrige}
\begin{tabular}{lrrrrrr}
\toprule
\textbf{Modèle} &
\textbf{MSE Train} &
\textbf{RMSE Train} &
\textbf{$R^2$ Train} &
\textbf{MSE Test} &
\textbf{RMSE Test} &
\textbf{$R^2$ Test} \\
\midrule
Random Forest & 0,04380 & 0,20927 & 0,9583 & 0,13866 & 0,37238 &
   0,8205 \\
XGBoost       & 0,01429 & 0,11956 & 0,9864 & 0,18860 & 0,43429 &
   0,7559 \\
SVR           & 0,18426 & 0,42925 & 0,8246 & 0,32597 & 0,57094 &
   0,5781 \\
\bottomrule
\end{tabular}
\end{table}

\subsection*{Interprétation}

Après correction, le pouvoir prédictif réel des indices
   satellite sur le
SPI se situe dans une fourchette réaliste ($R^2$ test entre 0,58
   et 0,82),
bien inférieure aux valeurs proches de 0,999 rapportées
   initialement, mais
scientifiquement significative.

Ces résultats montrent qu'une part substantielle de la variance
   du SPI
peut être expliquée par les seules données satellitaires, sans
   recours à
la mesure directe de précipitation. La hiérarchie entre modèles
   est
préservée : Random Forest demeure le plus performant sur le jeu
   de test
```

```
($R^2=0.8205$), suivi de XGBoost ($R^2=0.7559$), puis de SVR
($R^2=0.5781$).

L'écart entre les performances d'entraînement et de test est de
   0,1378
pour Random Forest, 0,2305 pour XGBoost et 0,2465 pour SVR, ce
   qui indique
un surapprentissage plus marqué pour XGBoost et SVR que pour
   Random Forest.

\section{Code source corrigé}

Le script Python correspondant doit être placé dans le répertoire
\texttt{code/}, par exemple sous le nom
\texttt{spi\_satellite\_leakage\_free.py}.

Pour l'inclure dans le document final :

\begin{verbatim}
\lstinputlisting[style=pythonstyle,
caption={Pipeline SPI corrigé et sans fuite de données}]
{code/spi_satellite_leakage_free.py}
\end{verbatim}

\section{Traçabilité des corrections}

Les scripts Python corrigés doivent être versionnés avec ce
   document dans
le même dépôt GitHub. Il est recommandé de conserver les scripts
   originaux
dans un répertoire distinct, par exemple \texttt{code/archive/},
   afin de
préserver la traçabilité des corrections méthodologiques.

\begin{itemize}
    \item pipeline SHI corrigé :
    \texttt{code/shi\_lucas2018\_leakage\_free.py} ;

    \item pipeline BiLSTM--CatBoost et test de Diebold--Mariano :
    \texttt{code/bilstm\_catboost\_dm\_leakage\_free.py} ;

    \item pipeline SPI corrigé :
    \texttt{code/spi\_satellite\_leakage\_free.py}.
\end{itemize}

\end{document}
"""

tex_path = base / "annexe_shi_spi_leakage_free.tex"
tex_path.write_text(tex, encoding="utf-8")
```

```
readme = """# Pipelines corrigés  SHI, BiLSTMCatBoost et SPI

Ce dépôt accompagne l'annexe méthodologique de la thèse et
   contient les
pipelines Python corrigés pour éviter les fuites de données
   (‘data leakage‘).

## Fichiers

- ‘annexe_shi_spi_leakage_free.tex‘ : document LaTeX de l'annexe.
- ‘code/shi_lucas2018_leakage_free.py‘ : pipeline SHI corrigé.
- ‘code/bilstm_catboost_dm_leakage_free.py‘ : comparaison
   BiLSTMCatBoost et test de DieboldMariano.
- ‘code/spi_satellite_leakage_free.py‘ : pipeline SPI corrigé.

## Principes méthodologiques

Les corrections portent notamment sur :

- séparation stricte entraînement/test ;
- ajustement des imputateurs et normaliseurs uniquement sur
   l'entraînement ;
- absence de réutilisation des composantes de la cible comme
   prédicteurs ;
- sélection des modèles/hyperparamètres sans utiliser le jeu de
   test ;
- validation temporelle ‘TimeSeriesSplit‘ pour les expériences
   temporelles ;
- exclusion des observations synthétiques des métriques finales
   lorsque le protocole l'exige ;
- audit automatique de fuite de données ;
- explicabilité SHAP réalisée après verrouillage du modèle final.

## Important

Les scripts Python doivent être ajoutés au répertoire ‘code/‘.
   Ils ne sont pas
reconstitués automatiquement à partir de l'annexe : il faut
   déposer ici les
versions corrigées effectivement exécutées et utilisées pour
   produire les
résultats rapportés dans la thèse.

## Compilation

Compiler ‘annexe_shi_spi_leakage_free.tex‘ avec XeLaTeX,
   LuaLaTeX ou pdfLaTeX
selon le préambule du projet principal.
"""
(base / "README.md").write_text(readme, encoding="utf-8")
```

```
code_dir = base / "code"
code_dir.mkdir(exist_ok=True)
(code_dir / "README.txt").write_text(
    "Déposer ici les scripts Python corrigés effectivement
       exécutés.\n"
    "Noms recommandés :\n"
    "- shi_lucas2018_leakage_free.py\n"
    "- bilstm_catboost_dm_leakage_free.py\n"
    "- spi_satellite_leakage_free.py\n",
    encoding="utf-8"
)

# Zip the package for easy GitHub preparation
import shutil
zip_path =
   shutil.make_archive("/mnt/data/annexe_shi_spi_leakage_free",
   "zip", base)

print(tex_path)
print(zip_path)
```

Listing 5 – Script de contrôle des risques de fuite de données

F. Organisation des codes sources

Les scripts utilisés pour reproduire les expériences sont regroupés dans le même répertoire que la présente annexe :

```
code/
|
|-- annexe_shi_spi_leakage_free.tex
|
|-- shi_lucas2018_leakage_free.py
|-- bilstm_catboost_dm_corrected_utf8.py
|-- spi_prediction_corrige.py
|-- version_false_start_corrige_sans_leakage.py
|-- Controler_data_leakage.py
```

Cette organisation permet d'associer directement chaque correction méthodologique à son implémentation informatique correspondante.

Les versions corrigées sont destinées à assurer une évaluation plus fiable de la capacité de généralisation des modèles et à faciliter la reproductibilité des résultats présentés dans la thèse.